



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación

Ingeniería en Software

PRÁCTICA EXPERIMENTAL III

Patrones Arquitecturales

Materia: Aplicaciones Web

Docente: Gleiston Guerrero Ulloa

Período Académico: SPA 2025-2026

Integrantes:

- Álava Alvarado Jean
- Mendoza Moreira Andy
- Reinoso Vélez Eduardo

Quevedo - Ecuador
Diciembre 2025

Índice

1. Introducción	2
1.1. Objetivo de la práctica	2
2. Glosario técnico comparativo	3
2.1. Aporte de Álava	3
2.2. Aporte de Mendoza	13
2.3. Aporte de Reinoso	19
3. Fichas técnicas de patrones arquitectónicos	29
3.1. Arquitectura en Capas	29
3.2. Arquitectura de Pizarra (Blackboard)	32
3.3. Arquitectura Dirigida por Eventos	34
3.4. Arquitectura Peer-to-Peer	36
3.5. Arquitectura Orientada a Servicios (SOA)	38
3.6. Modelo Vista Controlador (MVC)	40
4. Mapa conceptual comparativo	44
5. Conclusiones	44

Patrones arquitecturales

1. Introducción

El desarrollo de software contemporáneo exige una comprensión clara de las diferencias entre las aplicaciones de escritorio y las aplicaciones web, las cuales no se limitan al entorno de ejecución o a la interfaz de usuario, sino que se fundamentan principalmente en las decisiones arquitectónicas adoptadas durante el diseño del sistema. La arquitectura de software define la estructura esencial de un sistema, incluyendo sus componentes, las relaciones entre ellos y los principios que guían su evolución, convirtiéndose en un elemento clave para garantizar atributos de calidad como el rendimiento, la seguridad, la mantenibilidad y la escalabilidad [1], [2]. Comprender estos aspectos resulta indispensable para el desarrollo de soluciones de software robustas y alineadas con las necesidades actuales.

Los patrones arquitectónicos constituyen soluciones probadas para problemas recurrentes en el diseño de sistemas de información, permitiendo organizar de manera eficiente la lógica del sistema y separar responsabilidades. Patrones como la arquitectura en capas, el modelo vista-controlador (MVC) y los enfoques orientados a servicios se aplican tanto en aplicaciones de escritorio como en aplicaciones web, aunque con particularidades propias de cada plataforma y framework [3]. En este contexto, la presente práctica experimental se orienta a diferenciar la terminología y aplicación de estos patrones mediante la elaboración de un glosario técnico, fichas arquitectónicas y un mapa conceptual comparativo, con el fin de analizar cómo las decisiones arquitectónicas influyen en los atributos de calidad del software y en la experiencia del usuario final [1], [2].

1.1. Objetivo de la práctica

Diferenciar la terminología empleada en el desarrollo de aplicaciones de escritorio y aplicaciones web, identificando y caracterizando los patrones arquitectónicos más comunes mediante la elaboración de un glosario técnico, fichas arquitectónicas y un mapa conceptual comparativo.

2. Glosario técnico comparativo

En esta sección se presenta el glosario técnico compuesto por términos clave relacionados con arquitecturas de software, comunicación y redes, utilizados en aplicaciones de escritorio y aplicaciones web.

2.1. Aporte de Álava

Nº	Término	Aplicación en aplicaciones de escritorio	Aplicación en aplicaciones web
1	Acoplamiento	Mide qué tanto depende una clase de otra. En C#, se busca un bajo acoplamiento evitando el uso excesivo de la palabra clave new dentro de los formularios; en su lugar, se prefieren interfaces y patrones de inyección de dependencias nativos de .NET [4].	El framework reduce el acoplamiento drásticamente mediante su contenedor de Inversión de Control (IoC). Los objetos no crean sus dependencias, sino que el sistema se las inyecta [5].
2	Arquitectura cliente-servidor	Modelo de cliente grueso (Rich Client). La aplicación instalada en el PC (.exe) procesa la mayor parte de la lógica y la interfaz gráfica localmente, conectándose al servidor principalmente para extraer o guardar datos en el gestor de base de datos [6].	Modelo basado en cliente ligero. Spring Boot actúa como el servidor backend centralizando la lógica y seguridad; el cliente (navegador) solo se encarga de renderizar la respuesta (HTML/JSON) que recibe tras realizar una petición HTTP [7].
3	Arquitectura por capas	Organización lógica del código en proyectos separados dentro de una Solución (.sln). Se suele dividir en CapaPresentacion (WinForms), CapaNegocio (Clases lógicas) y CapaDatos (ADO.NET/EF Core), garantizando que el formulario no acceda directo a SQL [8].	Estructura estándar del proyecto dividida en paquetes: Controller (HTTP), Service (Lógica) y Repository (Datos). Spring gestiona el paso de datos entre estas capas automáticamente, promoviendo un código limpio y modular [8].

N°	Término	Aplicación en aplicaciones de escritorio	Aplicación en aplicaciones web
4	Binding de datos	Es la técnica de enlace que conecta automáticamente los controles de la interfaz (como un <code>TextBox</code>) con las propiedades de un objeto (<code>Producto.Nombre</code>). En Windows Forms, permite que si modificas el valor en el código, aparezca mágicamente en la caja de texto sin tener que escribir <code>txtNombre.Text = ...</code> manualmente [9].	En el contexto web, se refiere a la capacidad del framework para mapear automáticamente los datos que llegan en una petición HTTP (JSON o Formulario) a un objeto Java (POJO). Por ejemplo, usar la anotación <code>@RequestBody</code> convierte el JSON entrante directamente en una clase <code>Usuario</code> sin tener que parsearlo a mano [10].
5	Capa de datos	Es el nivel inferior de la arquitectura encargado exclusivamente de la comunicación con la base de datos (SQL Server, MySQL). En C# moderno, suele implementarse usando Entity Framework Core, el cual permite trabajar con datos usando objetos de C# en lugar de escribir consultas SQL manuales, facilitando el mantenimiento y la seguridad del código [11].	En este entorno, la capa de datos se implementa a través de Repositorios (Interfaces que extienden <code>JpaRepository</code>). Spring Data se encarga de generar automáticamente la implementación necesaria para realizar operaciones CRUD (Crear, Leer, Actualizar, Borrar) sin que el programador tenga que escribir el código de conexión JDBC manualmente [12].
6	Capa de negocio	También llamada BLL (Business Logic Layer). Es el núcleo donde residen las reglas, cálculos y validaciones del sistema (ej. “No permitir stock negativo”). En C#, se implementa mediante clases simples (POCOs) o servicios que reciben datos del Formulario, los validan y luego deciden si llamar a la capa de datos para guardar [13].	Se identifica con la anotación <code>@Service</code> . Aquí se encapsula la lógica empresarial y se gestionan las transacciones (usando <code>@Transactional</code>). Esta capa actúa de intermediario: el Controlador le pasa datos web, el Servicio aplica las reglas y luego llama al Repositorio para persistir la información [14].

N°	Término	Aplicación en aplicaciones de escritorio	Aplicación en aplicaciones web
7	Capa de presentación	Es la interfaz gráfica visible con la que interactúa el usuario (UI). En .NET (WinForms o WPF), esta capa se compone de Formularios y controles (botones, cajas de texto). Su responsabilidad se limita a capturar eventos (clics, tecleo) y mostrar datos, delegando cualquier procesamiento lógico complejo a la capa de negocio para no congelar la pantalla [13].	En la arquitectura web de Spring, esta capa está representada principalmente por los Controladores (<code>@RestController</code> o <code>@Controller</code>). Se encargan de interceptar las peticiones HTTP (GET, POST) que vienen del navegador (HTML/JS), interpretar qué recurso se pide y devolver la respuesta adecuada (JSON o una vista renderizada), actuando como la puerta de entrada a la aplicación [15].
8	Cohesión	Se refiere al grado en que los elementos de una clase (métodos y propiedades) están relacionados entre sí y trabajan para un único propósito lógico. En C#, un Formulario con alta cohesión es aquel que solo se encarga de gestionar la vista de una entidad específica (ej. <code>ClienteForm</code>), sin mezclar lógica de otros módulos como facturación o inventario [16].	En el backend, la cohesión dicta que una clase <code>@Service</code> debe enfocarse en una sola responsabilidad funcional. Por ejemplo, <code>FacturaService</code> debe contener solo métodos relacionados con facturas (calcular total, anular). Si empezara a tener métodos para “Crear Usuario” o “Enviar Email Marketing”, tendría baja cohesión, lo cual es una mala práctica [17].
9	Control de acceso a datos	Mecanismo de seguridad que restringe la manipulación de información según el perfil del usuario (Rol). En aplicaciones de escritorio, esto se suele implementar validando credenciales al inicio y luego habilitando/deshabilitando botones o menús en el Formulario según si el usuario es “Administrador” o “Cajero”, evitando que usuarios no autorizados disparen eventos de escritura en la base de datos [18].	Se gestiona principalmente a través del módulo Spring Security. Permite configurar filtros que interceptan cada petición HTTP antes de llegar a los datos. Se usan anotaciones como <code>@PreAuthorize("hasRole('ADMIN')")</code> sobre los métodos del Servicio o Repositorio para asegurar que solo los usuarios con el token y rol correcto puedan ejecutar consultas sensibles [19].

N°	Término	Aplicación en aplicaciones de escritorio	Aplicación en aplicaciones web
10	Control de navegación	Gestiona el flujo de pantallas dentro de la aplicación. En WinForms, esto implica instanciar y visualizar ventanas mediante métodos como <code>.Show()</code> (no modal) o <code>.ShowDialog()</code> (modal), y controlar el cierre o ocultamiento de formularios (<code>this.Close()</code>) para guiar al usuario a través de la tarea [20].	En el entorno web, la navegación se basa en Rutas (URLs). El controlador de Spring intercepta la URL solicitada (ej. <code>/productos/lista</code>) y determina qué vista (HTML) se debe renderizar o hacia qué otra ruta redirigir al navegador (usando <code>redirect:/...</code>), gestionando así el flujo de la experiencia del usuario [21].
11	Controlador	En aplicaciones de escritorio clásicas (WinForms), este rol suele estar implícito en el código del formulario (Code-behind). Los manejadores de eventos (como <code>button_Click</code>) actúan como controladores: capturan la acción del usuario, invocan a la lógica de negocio y actualizan la vista con el resultado [22].	En la arquitectura MVC web, es una clase explícita anotada con <code>@RestController</code> o <code>@Controller</code> . Su función es mapear las peticiones HTTP (URL) a métodos específicos, extraer los datos de la petición (Payload/Params) y delegar el procesamiento al servicio correspondiente [23].
12	Encapsulamiento	Es el principio de ocultar los detalles internos de funcionamiento y proteger el estado de un objeto. En C#, se aplica rigurosamente utilizando modificadores de acceso (<code>private</code> , <code>protected</code>) y exponiendo los datos únicamente a través de Propiedades (<code>public int Edad { get; set; }</code>), lo que permite validar el dato antes de asignarlo [24].	En el desarrollo con Spring, el encapsulamiento se manifiesta en el diseño de los Beans y DTOs (Data Transfer Objects). Se utilizan clases con atributos privados (<code>private</code>) y métodos accesorios (<code>getters/setters</code> , a menudo generados con la librería Lombok), asegurando que el controlador o la base de datos no manipulen los campos internos directamente sin pasar por los métodos definidos [25].

N°	Término	Aplicación en aplicaciones de escritorio	Aplicación en aplicaciones web
13	Escalabilidad estructural	Se refiere a la capacidad de la arquitectura para soportar el crecimiento del código y del equipo de desarrollo sin volverse inmanejable. En .NET, esto se logra dividiendo la Solución (.sln) en múltiples Proyectos de biblioteca de clases (.csproj), lo que permite que varios desarrolladores trabajen en módulos distintos (Facturación, Inventario) simultáneamente sin generar conflictos de código [26].	El modularidad se gestiona a través de herramientas de construcción como Maven o Gradle. Se crean módulos independientes que agrupan paquetes relacionados (Controller, Service, Repository de un dominio específico). Recientemente, proyectos como Spring Modulith promueven esta práctica para evitar que el código se vuelva inmanejable antes de saltar a microservicios [27].
14	Flujo de control	En este entorno, el flujo es dirigido por eventos (Event-Driven). El programa inicia un bucle principal (<code>Application.Run</code>) y permanece en estado de espera hasta que el usuario realiza una acción física (clic, tecleo). En ese momento, el control salta al método manejador del evento específico y retorna al estado de espera al terminar [28].	El flujo sigue el ciclo de Solicitud-Respuesta (Request-Response). El contenedor (Tomcat) recibe una petición HTTP, el <code>DispatcherServlet</code> de Spring toma el control y dirige el flujo linealmente hacia el Controlador adecuado, luego al Servicio y Repositorio, para finalmente retornar la respuesta procesada al navegador [29].
15	Flujo MVC	En el escritorio moderno (.NET MAUI/WPF), este flujo ha evolucionado al patrón MVVM (Model-View-ViewModel). Aquí, la Vista (Ventana XAML) no habla directo con el Modelo, sino que se conecta a un intermediario llamado View-Model mediante Binding. Esto desacopla totalmente la interfaz de la lógica, permitiendo cambiar el diseño visual sin tocar una línea de código C# [30].	El flujo es estricto y centralizado. El <code>DispatcherServlet</code> recibe la petición, busca el Controlador adecuado (<code>@Controller</code>), este procesa la solicitud, actualiza el Modelo (datos) y finalmente selecciona una Vista (HTML/JSON) para devolver al usuario. Es el corazón del funcionamiento de Spring Web [31].

N°	Término	Aplicación en aplicaciones de escritorio	Aplicación en aplicaciones web
16	Interfaz gráfica	En este entorno, la interfaz se construye mediante un Diseñador Visual (arrastrar y soltar controles como Button o DataGridView) que genera código automáticamente. La GUI es stateful (con estado): la ventana recuerda sus variables mientras está abierta y se comunica directamente con el sistema operativo Windows para dibujarse en pantalla [32].	En Spring Boot, la interfaz no es parte del código compilado de Java, sino que se sirve como contenido estático (HTML, CSS, JavaScript) o plantillas dinámicas (Thymeleaf). El navegador del cliente es quien interpreta y dibuja estos códigos para crear la experiencia visual, desacoplando la lógica del servidor de la presentación visual.
17	Mantenimiento del sistema	En el modelo de escritorio, el mantenimiento es descentralizado y costoso. Para corregir un error (bug) o agregar una funcionalidad, es necesario generar un nuevo instalador (.msi o .exe) y distribuirlo a cada computadora cliente. Si el usuario no actualiza, se queda con la versión obsoleta y propensa a fallos [33].	El mantenimiento es centralizado y transparente. Al desplegar una nueva versión del archivo .jar en el servidor, todos los usuarios acceden inmediatamente a la corrección o mejora con solo recargar su navegador, eliminando los problemas de versiones desincronizadas entre clientes [34].
18	Modelo	Representa los datos y la lógica de negocio pura. Se implementa mediante clases POCO (Plain Old CLR Objects), que son clases sencillas de C# con propiedades (<code>get;</code> <code>set;</code>). En arquitecturas modernas, estas clases suelen incluir anotaciones de validación (<code>[Required]</code>) para asegurar la integridad de los datos antes de que lleguen a la base de datos [35].	En Spring, el modelo se compone de POJOs (Plain Old Java Objects) que a menudo se mapean directamente a tablas de la base de datos usando la anotación <code>@Entity</code> de JPA. Estos objetos son la verdad única de la aplicación, definiendo qué datos existen y cómo se relacionan entre sí, independientemente de cómo se muestren en la web [36].

N°	Término	Aplicación en aplicaciones de escritorio	Aplicación en aplicaciones web
19	Modularidad	Es la capacidad de dividir la aplicación en bloques autónomos. En .NET, esto se logra físicamente separando el código en múltiples proyectos dentro de la misma solución (ej. Ventas.csproj, Inventario.csproj). Al compilar, cada proyecto genera su propio archivo .dll (Dynamic Link Library), lo que permite actualizar un módulo sin tener que recompilar toda la aplicación [37].	El modularidad se gestiona a través de herramientas de construcción como Maven o Gradle. Se crean módulos independientes que agrupan paquetes relacionados (Controller, Service, Repository de un dominio específico). Recientemente, proyectos como Spring Modulith promueven esta práctica para evitar que el código se vuelva inmanejable antes de saltar a microservicios [38].
20	Organización del proyecto	Se basa en el concepto de Solución (.sln), que actúa como un contenedor maestro. Dentro de la solución residen uno o más Proyectos (.csproj). Físicamente, la estructura típica incluye carpetas como Properties (configuración del ensamblado), References (librerías externas) y luego las carpetas definidas por el usuario (ej. Formularios, Clases) [37].	Sigue el Diseño de Directorios Estándar de Maven o Gradle. Todo el código fuente vive bajo <code>src/main/java</code> , organizado en paquetes inversos (ej. <code>com.miempresa.proyecto</code>). Los recursos no compilados (imágenes, HTML, configuración <code>application.properties</code>) se separan estrictamente en <code>src/main/resources</code> [38].
21	Patrón de diseño	Son soluciones reutilizables para problemas comunes de código. En aplicaciones de escritorio C#, uno de los más utilizados es el Singleton (para asegurar que solo exista una instancia de la conexión a la base de datos) o el Factory (para crear diferentes tipos de documentos o reportes dinámicamente según la selección del usuario) [39].	Spring Boot está construido sobre patrones. El más fundamental es la Inyección de Dependencias (una variante de Inversión de Control). Además, utiliza el patrón Singleton por defecto para todos sus Beans (componentes): el contenedor crea una sola instancia del Servicio y la reutiliza en cada petición HTTP para ahorrar memoria [40].

N°	Término	Aplicación en aplicaciones de escritorio	Aplicación en aplicaciones web
22	Persistencia	Es el proceso de guardar el estado de un objeto en un medio de almacenamiento permanente. En C# moderno, esto se delega casi siempre a un ORM (Object-Relational Mapper) como Entity Framework Core. El desarrollador trabaja con objetos (<code>context.Productos.Add(p)</code>) y el framework traduce esa acción en un INSERT de SQL automáticamente al llamar a <code>SaveChanges()</code> [41].	La persistencia se gestiona a través de la especificación JPA (Java Persistence API) implementada por Hibernate. Spring Data abstrae aún más esta complejidad: basta con extender una interfaz Repository para tener acceso inmediato a métodos de guardado, actualización y borrado sin escribir ni una línea de SQL [42].
23	Reglas de negocio	Son las restricciones lógicas que definen el funcionamiento de la empresa dentro del software. En C#, estas reglas no deben estar en el botón del formulario, sino encapsuladas en clases de la Capa de Negocio o utilizando librerías como FluentValidation para mantener el código ordenado y comprobable [43].	Estas reglas se implementan obligatoriamente dentro de las clases <code>@Service</code> . Antes de guardar cualquier dato, el servicio verifica que se cumplan todas las condiciones (validaciones, cálculos de impuestos, verificación de stock). Si alguna regla se rompe, el servicio lanza una excepción controlada para detener el proceso.
24	Renderizado de vistas	Es el proceso de dibujar los controles visuales en la pantalla del monitor. En WinForms, esto lo maneja el sistema operativo mediante eventos Paint. Cada vez que mueves o redimensionas una ventana, el código C# redibuja los píxeles de los botones y textos utilizando librerías gráficas como GDI+ o DirectX (en WPF) [44].	En el desarrollo web, el renderizado puede ser lado servidor (SSR), donde Spring usa motores como Thymeleaf para construir el HTML completo con datos antes de enviarlo; o lado cliente (CSR), donde Spring solo envía datos JSON y es el navegador (usando JavaScript/React) quien construye la página visualmente.

N°	Término	Aplicación en aplicaciones de escritorio	Aplicación en aplicaciones web
25	Reutilización de código	Se logra principalmente mediante la creación de Bibliotecas de Clases (.dll). Si tienes una lógica de cálculo de IVA que usas en varios programas, la colocas en un proyecto separado, lo compilas y luego referencias esa DLL en todas tus aplicaciones de escritorio. También se usan mucho los Controles de Usuario (UserControls) para reciclar diseños de botones o paneles en diferentes ventanas [45].	La reutilización se potencia mediante la Programación Orientada a Aspectos (AOP) y los Beans compartidos. Puedes crear un componente de Auditoría una sola vez e inyectarlo en docenas de servicios diferentes. Además, es común empaquetar funcionalidades comunes (como seguridad o manejo de logs) en librerías propias (Jars) que se importan con Maven en múltiples microservicios [46].
26	Responsabilidad única	Establece que una clase debe tener solo una razón para cambiar. En C#, es un error común poner lógica de base de datos dentro del código del botón (<code>Button.Click</code>). La práctica correcta es que el Formulario solo se encargue de mostrar, y delegue el cálculo o guardado a clases especializadas (Repositorios o Services), manteniendo el código limpio y fácil de arreglar [20].	Este principio se aplica estrictamente mediante las capas. El Controlador solo gestiona rutas HTTP (no calcula nada). El Servicio solo ejecuta lógica de negocio (no sabe de HTTP ni de SQL directo). El Repositorio solo habla con la base de datos. Si mezclas estas responsabilidades, rompes la arquitectura del framework y dificultas las pruebas unitarias [8].
27	Separación de responsabilidades	Principio arquitectónico donde cada parte del código tiene un trabajo único. En C#, esto significa no escribir consultas SQL dentro de los eventos del Formulario (<code>Form1.cs</code>). Lo correcto es mover esa lógica a una biblioteca de clases separada (Capa de Datos), dejando que el formulario solo se encargue de la visualización [47].	Este principio es la base de su arquitectura. Se divide estrictamente en: Controlador (gestiona peticiones web), Servicio (aplica la lógica de negocio) y Repositorio (habla con la BD). Esta separación permite cambiar la base de datos o la interfaz sin tener que reescribir todo el sistema [48].

N°	Término	Aplicación en aplicaciones de escritorio	Aplicación en aplicaciones web
28	Validación de datos	Es el proceso de asegurar que la información ingresada es correcta. En WinForms, se usa a menudo el componente <code>ErrorProvider</code> junto a la interfaz <code>IValidatableObject</code> . Esto permite mostrar iconos de error rojos junto a las cajas de texto si el usuario deja campos vacíos o introduce formatos incorrectos [49].	El estándar Jakarta Bean Validation. Se agregan anotaciones directamente en las clases del modelo, como <code>@NotNull</code> , <code>@Size(min=5)</code> o <code>@Email</code> . Si los datos que llegan del navegador no cumplen estas reglas, el framework rechaza la petición automáticamente antes de que llegue a la lógica de negocio [50].
29	Vista	Es el componente visual final. En aplicaciones de escritorio, la Vista es la ventana misma (Form), construida con código C# que define botones y etiquetas. Esta vista es inteligente (tiene código por detrás) y vive en la memoria de la computadora del usuario [20].	En la arquitectura web MVC, la vista es pasiva. Puede ser una plantilla HTML generada en el servidor (con Thymeleaf) o un JSON crudo que se envía al navegador. La vista no contiene lógica de negocio, su única función es presentar la información que le entregó el Controlador [51].
30	Dependencias entre capas	Define cómo se comunican los módulos. En .NET tradicional, las capas superiores (Presentación) agregan una Referencia de Proyecto hacia las capas inferiores (Negocio). Esto crea una dependencia estática que se resuelve en tiempo de compilación dentro de Visual Studio [52].	Spring gestiona esto mediante Inyección de Dependencias. Las capas no crean instancias de otras capas con <code>new</code> . En su lugar, el Controlador declara que necesita un Servicio (en su constructor), y el contenedor de Spring se encarga de conectar (inyectar) esa dependencia automáticamente en tiempo de ejecución [8].

2.2. Aporte de Mendoza

N°	Término	Aplicación en aplicaciones de escritorio	Aplicación en aplicaciones web
1	Framework	Un framework como .NET proporciona bibliotecas y herramientas que facilitan el desarrollo de aplicaciones de escritorio, permitiendo organizar el código y aplicar patrones como MVC [20], [32].	En Spring Boot, el framework Spring proporciona una estructura base para el desarrollo de aplicaciones web, facilitando la implementación del patrón MVC y el manejo de peticiones HTTP [8], [15].
2	Acceso a datos	En aplicaciones de escritorio en C#, el acceso a datos se realiza desde el Modelo mediante tecnologías como ADO.NET o Entity Framework, permitiendo obtener y persistir información en bases de datos manteniendo separada la lógica de negocio [11], [41].	En Spring Boot, el acceso a datos se gestiona en el Modelo a través de repositorios y tecnologías como JPA/Hibernate, facilitando la interacción con bases de datos y manteniendo la separación de responsabilidades del patrón MVC [40], [42].
3	Aplicación de escritorio	Una aplicación de escritorio es un software que se ejecuta localmente en el sistema operativo del usuario, con interfaz gráfica propia, donde el patrón MVC permite separar la lógica del negocio, la presentación y el control de eventos [20], [32].	En Spring Boot no se desarrollan aplicaciones de escritorio, sino aplicaciones web; sin embargo, el patrón MVC se aplica de forma similar para separar la lógica del negocio, el control de peticiones y la presentación de la información [53], [54].
4	Aplicación web	En C#, el desarrollo web se realiza mediante frameworks como ASP.NET, donde el patrón MVC permite estructurar la aplicación separando la lógica del negocio, el control de peticiones y la presentación [13], [55].	En Spring Boot, una aplicación web es un sistema que se ejecuta en un servidor y es accedido mediante un navegador, utilizando el patrón MVC para gestionar peticiones HTTP, lógica del negocio y vistas dinámicas [8], [56].

N°	Término	Aplicación en aplicaciones de escritorio	Aplicación en aplicaciones web
5	Arquitectura web	En C#, la arquitectura web se implementa principalmente con ASP.NET MVC, donde la aplicación se organiza en capas para manejar lógica de negocio, control de peticiones y presentación [26], [55].	En Spring Boot, la arquitectura web se basa en el patrón MVC y en una estructura cliente-servidor, donde el controlador gestiona peticiones HTTP, el modelo contiene la lógica y la vista presenta la información al usuario [15], [57].
6	Ciclo de vida de la aplicación	En C#, el ciclo de vida de una aplicación de escritorio comprende la inicialización, ejecución y cierre del programa, donde el patrón MVC ayuda a organizar las responsabilidades durante cada etapa [20], [28].	En Spring Boot, el ciclo de vida de la aplicación web incluye el arranque del servidor, la gestión de peticiones HTTP y la finalización del contexto de la aplicación, siguiendo la estructura del patrón MVC [21], [58].
7	Cliente	En aplicaciones de escritorio en C#, el cliente es el propio programa instalado en el equipo del usuario, desde donde se ejecuta la interfaz y se consumen los datos del sistema siguiendo la estructura MVC [6], [32].	En Spring Boot, el cliente es generalmente el navegador web u otra aplicación que envía peticiones HTTP al servidor, solicitando recursos que son procesados mediante el patrón MVC [7], [15].
8	Compilación	En C#, la compilación es el proceso mediante el cual el código fuente se transforma en un ejecutable (.exe) o ensamblado, permitiendo que la aplicación de escritorio se ejecute en el sistema operativo [20], [28].	En Spring Boot, la compilación convierte el código Java en archivos bytecode y posteriormente en un archivo JAR, que puede ejecutarse en un servidor para desplegar la aplicación web [36], [38].
9	Configuración del sistema	En C#, la configuración del sistema se define mediante archivos como <code>app.config</code> o <code>web.config</code> , donde se establecen parámetros de ejecución, conexión a datos y comportamiento de la aplicación de escritorio [18], [37].	En Spring Boot, la configuración del sistema se gestiona principalmente con el archivo <code>application.properties</code> o <code>application.yml</code> , donde se definen puertos, conexiones a bases de datos y parámetros del entorno de la aplicación web [8], [38].

N°	Término	Aplicación en aplicaciones de escritorio	Aplicación en aplicaciones web
10	Controlador web	En C#, un controlador web se utiliza en frameworks como ASP.NET MVC para recibir solicitudes del usuario, ejecutar la lógica correspondiente y devolver una vista o respuesta [13], [55].	En Spring Boot, el controlador web es una clase anotada con <code>@Controller</code> o <code>@RestController</code> que maneja las peticiones HTTP, interactúa con el modelo y define la respuesta enviada al cliente [15], [59].
11	Control de sesiones	En C#, el control de sesiones se maneja principalmente mediante variables en memoria o mecanismos propios del framework para mantener el estado del usuario durante la ejecución de la aplicación de escritorio [18], [20].	En Spring Boot, el control de sesiones se gestiona a través de sesiones HTTP, permitiendo mantener información del usuario entre múltiples peticiones web dentro del patrón MVC [21], [56].
12	Despliegue	En C#, el despliegue de una aplicación de escritorio consiste en la distribución del ejecutable o instalador para que el usuario final lo ejecute localmente en su sistema [33], [37].	En Spring Boot, el despliegue implica publicar el archivo JAR o WAR en un servidor o entorno en la nube para que la aplicación web esté disponible a través de internet [34], [38].
13	Ejecución local	La aplicación se ejecuta directamente en el sistema operativo del usuario, utilizando los recursos locales del equipo sin depender de un servidor externo [28], [32].	La aplicación puede ejecutarse en un entorno local durante el desarrollo, pero está diseñada para funcionar en un servidor que atiende múltiples solicitudes de usuarios de forma simultánea [8], [21].
14	Endpoint	El acceso a servicios externos se realiza mediante puntos definidos de comunicación, que permiten enviar y recibir información cuando la aplicación interactúa con sistemas remotos [18], [55].	Las funcionalidades del sistema se exponen a través de direcciones web específicas que permiten a los clientes enviar solicitudes y recibir respuestas [15], [57].

N°	Término	Aplicación en aplicaciones de escritorio	Aplicación en aplicaciones web
15	Entorno de ejecución	El código fuente se ejecuta dentro del entorno .NET, el cual se encarga de administrar memoria, controlar hilos de ejecución y garantizar el correcto funcionamiento del programa durante su uso [20], [28].	La ejecución se realiza dentro de la Máquina Virtual de Java, encargada de administrar memoria, procesos y recursos del servidor donde se despliega la aplicación [25], [36].
16	Enrutamiento	La dirección de acciones internas se gestiona principalmente a través de eventos y componentes de la interfaz, permitiendo que las solicitudes del usuario sean procesadas de forma ordenada [20], [22].	Las solicitudes recibidas son dirigidas a métodos específicos mediante rutas definidas, lo que permite una organización clara de las funcionalidades del sistema [15], [60].
17	Estado de la aplicación	La información y el estado del sistema se mantienen en memoria mientras la aplicación permanece abierta, conservando datos necesarios para la continuidad de las operaciones del usuario [18], [28].	El sistema puede manejar información del usuario entre solicitudes o diseñarse para no conservar estado, dependiendo de los requerimientos de la aplicación [21], [56].
18	Stateful	El sistema conserva valores, configuraciones y datos temporales durante toda la ejecución, permitiendo que la aplicación recuerde acciones previas hasta que se cierre [20], [32].	La información del usuario puede mantenerse entre peticiones mediante mecanismos de sesión, permitiendo una experiencia continua durante la navegación [21], [58].
19	Inyección de dependencias	La creación y gestión de objetos se organiza de manera que las dependencias entre componentes estén desacopladas, facilitando la modificación y el mantenimiento del código [4], [13].	El framework gestiona automáticamente la creación y vinculación de componentes, reduciendo el acoplamiento y mejorando la organización del código [8], [40].
20	Logs	Se registran eventos relevantes, errores y acciones del sistema con el fin de analizar el comportamiento de la aplicación y facilitar tareas de depuración [18], [20].	Se registran eventos, solicitudes y errores del sistema con el fin de monitorear el comportamiento de la aplicación y detectar fallos [21], [46].

N°	Término	Aplicación en aplicaciones de escritorio	Aplicación en aplicaciones web
21	Manejo de errores	Los errores se controlan mediante mecanismos que permiten capturar fallos, evitando cierres inesperados y asegurando la estabilidad del sistema [20], [28].	Los errores se gestionan de forma centralizada, permitiendo devolver respuestas controladas al cliente y evitar fallos inesperados [15], [56].
22	Mantenibilidad	La estructura del software facilita su mantenimiento gracias a la separación clara entre lógica de negocio, interfaz gráfica y control de eventos [58], [61].	La arquitectura modular y el uso del patrón MVC facilitan la modificación, ampliación y mantenimiento del sistema a largo plazo [58], [59].
23	Navegador	La interacción con el usuario se realiza mediante una interfaz gráfica propia, sin necesidad de utilizar un navegador web para acceder a las funcionalidades del sistema [20], [32].	El acceso a la aplicación se realiza a través de un navegador web, el cual actúa como intermediario entre el usuario y el servidor [7], [57].
24	Petición HTTP	La comunicación mediante protocolos web se emplea únicamente cuando es necesario consumir servicios externos o intercambiar información con otros sistemas [18], [55].	Las solicitudes enviadas por el cliente permiten solicitar información o ejecutar acciones específicas dentro del sistema [15], [56].
25	Respuesta HTTP	La información recibida como resultado de una comunicación externa es procesada y utilizada por la aplicación para continuar con su funcionamiento normal [18], [55].	El servidor procesa las solicitudes recibidas y devuelve información estructurada como respuesta al cliente [15], [57].
26	Runtime	El entorno de ejecución proporciona los servicios necesarios para interpretar y ejecutar el código compilado, garantizando la correcta gestión de recursos del sistema [20], [28].	La ejecución del sistema depende del entorno proporcionado por la JVM, que permite el funcionamiento de la aplicación web [25], [36].

N°	Término	Aplicación en aplicaciones de escritorio	Aplicación en aplicaciones web
27	Seguridad de la aplicación	La protección del sistema se aplica mediante validaciones, control de accesos y restricciones que evitan el uso indebido de la aplicación [18], [58].	La seguridad se gestiona mediante mecanismos de autenticación, autorización y protección de recursos expuestos [19], [58].
28	Servidor web	No se requiere un servidor web para su funcionamiento, ya que todas las operaciones se ejecutan de manera local en el equipo del usuario [6], [32].	El servidor web es el componente encargado de recibir las solicitudes, procesarlas y enviar las respuestas correspondientes [7], [21].
29	Sesión	La información del usuario se mantiene disponible mientras la aplicación está en ejecución, perdiéndose al cerrarse el programa [20], [28].	La información del usuario se conserva entre solicitudes mediante mecanismos de sesión controlados por el servidor [21], [56].
30	Stateless	El funcionamiento se basa en mantener datos y estados activos durante la ejecución, lo que permite una interacción continua y persistente [18], [32].	Cada solicitud se procesa de manera independiente, sin conservar información previa, permitiendo mayor escalabilidad del sistema [56], [60].

2.3. Aporte de Reinoso

N°	Término	Aplicación en aplicaciones de escritorio	Aplicación en aplicaciones web
1	Acoplamiento débil	Principio de diseño que minimiza las dependencias entre módulos de software, donde los componentes dependen de abstracciones mediante interfaces bien definidas en lugar de implementaciones concretas [62]. Se implementa mediante inyección de dependencias por constructor que permite sustituir componentes sin modificar el código dependiente, facilitando el mantenimiento y las pruebas unitarias.	En arquitecturas web como Spring Boot, el acoplamiento débil se logra mediante separación en capas y servicios RESTful que exponen interfaces estandarizadas, permitiendo que los componentes se comuniquen a través de contratos de API definidos [3]. Los microservicios representan este principio al máximo nivel, posibilitando el despliegue y escalado independiente de servicios.
2	Arquitectura de software	La arquitectura de software se materializa mediante patrones como la arquitectura en capas (Presentation Layer, Business Logic Layer, Data Access Layer) y el patrón MVC/MVVM, comúnmente implementados con tecnologías como .NET Framework, WPF y Entity Framework para gestionar la persistencia de datos y la separación de responsabilidades[1].	Emplea enfoques como microservicios, arquitectura en capas y patrones REST API, implementados frecuentemente con Spring Boot que ofrece autoconfiguración, servidores embebidos (Tomcat, Jetty) y soporte nativo para crear servicios escalables, desacoplados y listos para producción mediante Spring Cloud[1][63].
3	Arquitectura distribuida	La arquitectura distribuida se implementa mediante comunicación entre procesos con tecnologías como .NET Remoting, Windows Communication Foundation (WCF) y gRPC, donde los componentes distribuidos en una red local emplean protocolos de llamada a procedimientos remotos (RPC) para coordinar operaciones manteniendo estado de sesión y conexiones persistentes[64].	Se materializa mediante microservicios con Spring Boot, empleando APIs REST sobre HTTP, mensajería asíncrona con Apache Kafka o RabbitMQ, y patrones como SAGA para transacciones distribuidas, facilitando escalabilidad horizontal, tolerancia a fallos y despliegue independiente de servicios mediante contenedores y orquestadores como Kubernetes[64][65].

N°	Término	Aplicación en aplicaciones de escritorio	Aplicación en aplicaciones web
4	Arquitectura monolítica	Se implementa mediante frameworks como .NET con Windows Forms o WPF, donde la interfaz gráfica, la lógica empresarial y el acceso a datos locales residen en un único ejecutable compilado que opera de manera autónoma, facilitando el despliegue simple mediante instaladores pero requiriendo recompilación completa ante cualquier modificación[66].	Con Spring Boot estructura la aplicación en capas lógicas (presentación, negocio, persistencia) dentro de un único archivo JAR o WAR desplegable, aprovechando servidores embebidos como Tomcat y gestionando todas las funcionalidades mediante un solo proceso, lo cual simplifica el desarrollo y testing iniciales pero puede presentar limitaciones de escalabilidad cuando aumenta la complejidad del sistema[67][66].
5	Bus de eventos	Se implementa mediante patrones in-process como MediatR o Prism EventAggregator, donde los eventos se publican y consumen dentro del mismo espacio de memoria de la aplicación para coordinar interacciones entre componentes de la interfaz gráfica (WPF, Windows Forms) y la lógica de negocio, manteniendo bajo acoplamiento mediante delegados y handlers de eventos[66].	Se materializa mediante brokers de mensajería externos como Apache Kafka, RabbitMQ o AWS EventBridge que gestionan la publicación y consumo de eventos entre microservicios distribuidos, empleando topics y exchanges para enrutamiento basado en reglas, garantizando entrega asíncrona confiable, escalabilidad horizontal y tolerancia a fallos mediante colas persistentes y mecanismos de retry[68][66].
6	Blackboard	Se implementa mediante una clase centralizada que actúa como repositorio compartido de datos y estado de la solución, con múltiples componentes especializados (implementados como clases independientes o servicios) que leen, analizan y escriben información en el blackboard de manera asíncrona, coordinados por un scheduler que determina qué componente debe activarse según el estado actual del problema[69].	Se materializa utilizando estructuras de datos compartidas en Redis o bases de datos in-memory, donde microservicios independientes actúan como fuentes de conocimiento que consultan y actualizan el blackboard mediante APIs REST o mensajería asíncrona, con un servicio coordinador implementado como componente Spring que gestiona la activación de servicios según reglas de dominio y el estado evolutivo de la solución almacenada en el blackboard[69][70].

N°	Término	Aplicación en aplicaciones de escritorio	Aplicación en aplicaciones web
7	Comunicación asíncrona	Se implementa mediante el patrón <code>async/await</code> con la clase <code>Task</code> , donde métodos marcados como <code>async</code> permiten operaciones no bloqueantes de entrada/salida (acceso a archivos, bases de datos, servicios remotos) liberando el thread de la interfaz gráfica para mantener la responsividad de aplicaciones WPF o Windows Forms mientras se ejecutan operaciones en background[68].	Se materializa mediante Spring WebFlux con programación reactiva basada en Project Reactor, implementando flujos no bloqueantes con tipos <code>Mono</code> y <code>Flux</code> que permiten procesar streams de datos asíncronamente, y soportar backpressure para controlar la velocidad de producción y consumo de mensajes en arquitecturas de microservicios escalables[64][68].
8	Componentes distribuidos	Se implementan mediante tecnologías como Windows Communication Foundation (WCF), .NET Remoting y COM+ para crear objetos remotables que exponen servicios a través de diversos protocolos de transporte (TCP, Named Pipes, HTTP), permitiendo que aplicaciones cliente de escritorio invoquen métodos en componentes alojados en servidores remotos mediante proxies generados automáticamente y serialización de objetos[65].	Se materializan como microservicios independientes empaquetados en contenedores Docker que exponen APIs REST o implementan comunicación mediante gRPC, registrándose en servicios de descubrimiento como Eureka o Consul para habilitar invocación remota transparente entre componentes, gestionando transacciones distribuidas mediante patrones SAGA y garantizando resiliencia con circuit breakers implementados por Spring Cloud[65][71].
9	Control de flujo	Se implementa mediante mecanismos de sincronización como <code>Monitor</code> , <code>Semaphore</code> , <code>ManualResetEvent</code> y <code>SemaphoreSlim</code> que coordinan el acceso a recursos compartidos entre threads en una aplicación WPF o Windows Forms, permitiendo que operaciones asincrónicas con <code>Task</code> y <code>async/await</code> no bloqueen indefinidamente el thread de interfaz gráfica mientras se espera la finalización de operaciones I/O o procesamiento en segundo plano[66].	Se materializa mediante la especificación Reactive Streams implementada por Project Reactor, que proporciona backpressure no bloqueante donde consumidores pueden solicitar explícitamente el número de elementos a procesar mediante <code>request()</code> y operadores como <code>limitRate()</code> , <code>onBackpressureBuffer()</code> y <code>throttle()</code> regulan la emisión de datos desde productores [64][66].

N°	Término	Aplicación en aplicaciones de escritorio	Aplicación en aplicaciones web
10	Contrato de servicio	Se implementa mediante el atributo ServiceContract en interfaces o clases de Windows Communication Foundation (WCF), donde cada operación se marca con OperationContract para definir métodos invocables remotamente, especificando el namespace, protección de datos mediante ProtectionLevel, y el comportamiento de sesiones[72].	Se materializa mediante especificaciones OpenAPI 3.0 documentadas con Swagger, donde anotaciones como @Tag y @Schema definen endpoints REST, esquemas de datos JSON, códigos HTTP y políticas de autenticación, generando documentación interactiva que sirve como contrato para generación automática de clientes, validación de contratos en CI/CD y facilitación de estrategias contract-first s[72][73].
11	Core del sistema	Se implementa mediante una capa Domain o Business Logic Layer que contiene entidades POCO (Plain Old CLR Objects), servicios de dominio y interfaces de repositorios, construida como un proyecto independiente de .NET Standard o .NET Core sin referencias a Entity Framework, WPF o tecnologías específicas, permitiendo que la lógica empresarial sea testeable y reutilizable independientemente de la interfaz gráfica o mecanismo de persistencia[74].	Se materializa en el módulo domain dentro de arquitecturas hexagonales o de cebolla, conteniendo agregados, servicios de dominio e interfaces de puertos que definen contratos sin anotaciones de Spring, mientras que las capas implementan casos de uso y adaptadores respectivamente, garantizando que la lógica de negocio permanezca testeable mediante JUnit sin contenedor Spring y portable a diferentes contextos de ejecución[74][3].
12	Desacoplamiento	Se implementa mediante patrones como Dependency Injection (inyección de dependencias) a través de contenedores como Microsoft.Extensions.DependencyInjection, interfaces segregadas que definen contratos explícitos entre capas, y patrones como Facade y Repository para abstraer la complejidad de acceso a datos y servicios externos[75].	Se materializa mediante inyección de dependencias automática a través de anotaciones @Autowired o constructores, arquitectura de puertos y adaptadores donde el dominio no depende de frameworks Spring, permitiendo que servicios distribuidos evolucionen de manera independiente sin romper contratos establecidos[3][75].

N°	Término	Aplicación en aplicaciones de escritorio	Aplicación en aplicaciones web
13	Evolutividad del sistema	Se implementa mediante arquitectura modular en capas o basada en dominios donde cada módulo se empaques como un proyecto .NET independiente con responsabilidades claras, versionado de NuGet para actualizar componentes sin afectar otros, inyección de dependencias que desacopla implementaciones de abstracciones permitiendo reemplazar servicios, y pruebas automatizadas que validan que cambios no rompan contratos [76].	Se logra mediante microservicios con límites definidos por dominios de negocio en lugar de capas técnicas, donde cada servicio puede evolucionar en versionado independiente, implementar patrones como Circuit Breaker para manejar cambios sin cascadas de fallos, configuración centralizada para adaptarse sin redeploy, y fitness functions del ecosistema devops que validen que cambios cumplan objetivos de escalabilidad y seguridad [76][77].
14	Flujo de información	Se estructura mediante arquitectura de capas donde los datos viajan desde la capa de presentación hacia la capa de negocio y luego hacia la capa de persistencia, empleando patrones como Repository y DTO para transformar datos entre capas, asegurando que información se valida, autoriza y transforma en cada punto de paso mediante inyección de dependencias que permite rastrear y reemplazar fuentes de datos[78].	Implementa arquitectura pipe-and-filter mediante streams reactivos en Spring WebFlux donde datos fluyen asincrónicamente desde clientes mediante APIs REST, permitiendo procesamiento paralelo, desacoplamiento temporal y monitoreo centralizado del flujo de datos desde fuentes hasta destinos finales mediante herramientas de observabilidad y tracing distribuido[1][78].
15	Gestión de eventos	Se implementa mediante el patrón Observer nativo de C# con eventos y delegates, donde componentes exponen propiedades de evento que otros componentes pueden suscribirse para reaccionar a cambios, complementado con patrones como Mediator que centraliza la gestión de eventos en memoria, permitiendo comunicación desacoplada entre ventanas, controles y servicios de negocio sin referencias directas entre productores y consumidores[79].	Se materializa mediante Spring Events para eventos locales dentro de un servicio, Event Sourcing para persistencia de eventos inmutables que representan cambios de dominio, mensajería mediante Kafka para enrutamiento entre microservicios, garantizando auditabilidad total, time-travel debugging y consistencia eventual en arquitecturas distribuidas de microservicios escalables[68][79].

N°	Término	Aplicación en aplicaciones de escritorio	Aplicación en aplicaciones web
16	Integración de sistemas	Se implementa mediante patrones punto-a-punto para conexiones simples o hub-and-spoke centralizado con middleware que rutea comunicación entre componentes, utilizando WCF para servicios remotos, COM/DCOM para componentes locales, o importación/exportación de archivos para intercambio batch, permitiendo integrar funcionalidades nuevas sin rediseño completo[72].	Evoluciona hacia Enterprise Service Bus (ESB) o patrones API-first con microservicios exponiendo APIs REST, implementando orquestación mediante Spring Integration, transformación de datos, enrutamiento inteligente y tolerancia a fallos con reintentos y dead-letter queues, logrando coexistencia de sistemas heredados con microservicios modernos[68][72].
17	Interoperabilidad	Se implementa mediante SOAP y WSDL en WCF para servicios remotos, serialización XML o JSON para intercambio con sistemas heterogéneos, e interfaces bien definidas que abstraen implementación permitiendo que aplicaciones .NET se comuniquen con servicios Java, Python u otros mediante protocolos neutrales como HTTP/REST[72].	Se logra mediante APIs REST con estándares HTTP, OpenAPI/Swagger para documentación independiente, formatos universales (JSON, XML), y versionado de API que permite evolución sin romper contratos, facilitando coexistencia de microservicios en Spring Boot, Node.js o .NET mediante interfaces estandarizadas[65][72].
18	Latencia	Se gestiona mediante procesamiento asíncrono con async/await evitando bloqueo de threads, paralelismo con Task Parallel Library para múltiples procesadores, caching para reducir acceso a datos lentos, y optimización de operaciones críticas en WPF o Windows Forms minimizando overhead, logrando tiempos de respuesta de milisegundos[80].	Se reduce mediante arquitectura reactiva con Spring Web-Flux no bloqueante, distribución geográfica con CDN acercando contenido a usuarios, caches distribuidos como Redis, balanceo de carga evitando cuellos de botella, compresión de payloads, y monitoreo continuo para identificar componentes que introducen latencia significativa[1][80].

N°	Término	Aplicación en aplicaciones de escritorio	Aplicación en aplicaciones web
19	Mensajería	Se implementa mediante Azure Service Bus o MSMQ para mensajes persistentes con patrones punto-a-punto mediante queues y pub-sub mediante topics, permitiendo comunicación asincrónica con ServiceBusClient para envío, deadletter queues para errores, y ServiceBusProcessor para consumo con entrega garantizada at-least-once[81].	Se materializa mediante Kafka o RabbitMQ con Spring Cloud Stream, donde servicios publican eventos a topics que otros suscriben mediante listeners asincrónicos, implementando Saga Pattern para transacciones distribuidas, outbox pattern para atomicidad, y Dead Letter Topic para mensajes no procesables[68][81].
20	Middleware	Se implementa mediante Entity Framework para persistencia desacoplando de bases de datos específicas, inyección de dependencias con Microsoft.Extensions.DependencyInjection y en ASP.NET Core mediante pipelines que procesan requests con filtros y handlers, implementando cross-cutting concerns como autenticación, validación y logging centralizadamente[82].	Se materializa mediante Spring Framework con inyección de dependencias, Spring Cloud para patrones distribuidos (service discovery, circuit breakers, config management), y filtros/interceptadores procesando requests HTTP, complementado con Kafka o RabbitMQ para comunicación asincrónica y coordinación de eventos entre servicios[65][82].
21	Orquestación	Se implementa mediante coordinadores centrales orquestando llamadas a componentes locales, usando Mediator para centralizar coordinación entre módulos, o Windows Workflow Foundation para procesos complejos con ramificaciones condicionales, reintentos automáticos y compensación ante fallos, permitiendo composición declarativa de tareas interdependientes[81].	Se materializa mediante servicio orquestador invocando otros servicios en secuencia vía REST o gRPC, implementando Saga orquestado donde el coordinador dispara compensaciones ante fallos, usando Temporal o Camunda para workflows complejos, y empleando reintentos exponenciales, circuit breakers y timeouts para confiabilidad[81][1].

N°	Término	Aplicación en aplicaciones de escritorio	Aplicación en aplicaciones web
22	Orientación a servicios	Se implementa mediante WCF definiendo ServiceContract sobre interfaces para contratos explícitos, usando SOAP sobre HTTP o protocolos binarios, aplicando principios de Erl como loose coupling mediante interfaces abstractas, service abstraction ocultando implementación, y service autonomy donde cada servicio controla su lógica independientemente[1].	Evoluciona hacia microservicios adoptando principios SOA enfatizando desacoplamiento más fuerte, independencia de despliegue y límites definidos por dominios de negocio, implementando service discovery con Eureka o Kubernetes, service reusability mediante APIs versionadas, logrando escalabilidad donde equipos evolucionan servicios independientemente[72][1].
23	Patrón arquitectónico	Incluyen arquitectura en capas organizando en presentación, negocio y persistencia para separación de responsabilidades, MVC/MVVM estructurando interfaces con modelos separados de vistas, y Microkernel permitiendo plugins independientes conectados a núcleo central, permitiendo cambios en una capa sin afectar otros componentes[1].	Incluyen microservicios descomponiendo en servicios independientes deployables con responsabilidades limitadas, event-driven comunicando mediante eventos asincrónicos desacoplando productor de consumidores, y arquitectura hexagonal separando dominio de adaptadores para testabilidad, facilitando escalabilidad horizontal y evolución independiente[78][1].
24	Procesamiento distribuido	Se implementa mediante Task Parallel Library abstrayendo threading permitiendo paralelizar con Parallel.For y Parallel.ForEach, coordinando tasks que explotan procesadores multi-núcleo locales para operaciones compute-intensive, complementado con mecanismos de cancelación, continuaciones y agregación garantizando sincronización eficiente en memoria compartida[83].	Se logra mediante Apache Spark particionando datos en RDDs distribuidos entre ejecutores, implementando transformaciones map, reduce, filter operando en paralelo sobre particiones, y patrones MapReduce para agregación de resultados parciales, facilitando procesamiento masivo mediante paralelismo distribuido transparente con tolerancia automática a fallos[65][83].

N°	Término	Aplicación en aplicaciones de escritorio	Aplicación en aplicaciones web
25	Publicador	Se implementa mediante Observer de C# donde componentes exponen eventos que otros suscriben, o MediatR como publicador centralizado de notificaciones en memoria, permitiendo que servicios publiquen eventos de dominio sin acoplamientos directos hacia múltiples handlers independientes, facilitando testabilidad y cambios localizados[69].	Se materializa mediante Spring Events para eventos locales, o Kafka como publicador distribuido donde servicios publican domain events a topics que múltiples suscriptores consultan independientemente, permitiendo que publicadores escalables entreguen eventos sin esperar consumidores, facilitando event sourcing donde el historial de eventos constituye la fuente de verdad[68][69].
26	Repositorio central	Se implementa mediante clases encapsulando Entity Framework Core o ADO.NET, exponiendo métodos GetById(), GetAll(), Add(), Update() y Delete() sobre agregados de dominio, permitiendo acceso a datos sin conocer detalles de base de datos, facilitando testabilidad mediante inyección de interfaces mock y migración transparente entre tecnologías de persistencia[3].	Se materializa mediante Spring Data proporcionando interfaces JpaRepository o CrudRepository extendiendo funcionalidades CRUD automáticamente, permitiendo comunicación con persistencia mediante métodos de alto nivel sin SQL manual, soportando múltiples backends (SQL, NoSQL, Cassandra), y facilitando consultas derivadas mediante anotaciones[74][3].
27	Servicio web	Se implementan mediante SOAP y WSDL en WCF generando proxies automáticamente, permitiendo invocar métodos remotos como locales, serializando con XML, manejando seguridad con WS-Security, y soportando transacciones distribuidas con WS-AtomicTransaction, aunque modernamente se prefiere REST con JSON para simplificar integración[72].	Evolucionan hacia REST APIs implementando Arquitectura Orientada a Recursos describiendo funcionalidad mediante recursos HTTP accesibles por URIs, donde operaciones GET/POST/PUT/DELETE manipulan representaciones JSON, con caching HTTP, escalabilidad mediante statelessness, y composición de microservicios exponiendo APIs simplificadas[81][72].

N°	Término	Aplicación en aplicaciones de escritorio	Aplicación en aplicaciones web
28	Suscriptor	Se implementa mediante handlers registrados con publicadores en WPF o Windows Forms, o suscriptores MediatR implementando INotificationHandler para reaccionar a notificaciones de dominio centralmente, permitiendo que múltiples handlers procesen un evento independientemente sin acoplamiento directo, facilitando testabilidad y adición de comportamientos[69].	Se materializa mediante listeners registrados en topics Kafka o exchanges RabbitMQ para consumir eventos publicados, implementando handler reactivo procesando eventos asincrónicamente, logrando event-driven donde suscriptores se adicionan dinámicamente sin conocimiento del publicador, facilitando escalabilidad horizontal y reactividad[68][69].
29	Tolerancia a fallos	Se implementa mediante reintentos automáticos con backoff exponencial para operaciones transitorias, timeouts evitando bloqueos indefinidos, replicación de datos críticos, manejo robusto de excepciones diferenciando fallos recuperables e irrecuperables, permitiendo degradación elegante donde funcionalidades no críticas se deshabilitan temporalmente[1].	Se logra mediante Circuit Breaker deteniendo solicitudes a servicios fallidos evitando cascada de fallos, implementados con Hystrix o Resilience4j, complementado con retry logic, fallback methods, bulkhead pattern mediante threadpools, y timeouts limitando recursos bloqueados, replicación mediante sharding, logrando alta disponibilidad[1][65].
30	Escalabilidad	Se logra mediante escalado vertical optimizando uso eficiente de recursos locales, implementando caching agresivo reduciendo acceso a datos lentos, paralelización con Task Parallel Library aprovechando multinúcleo, y arquitectura stateless donde cada instancia sirve requests independientemente[1].	Se alcanza mediante microservicios permitiendo escalado horizontal independiente según demanda específica, implementando statelessness donde instancias se agregan/remueven dinámicamente, load balancing distribuyendo traffic, caching distribuido con Redis, database sharding particionando datos horizontalmente, y orquestación con Kubernetes[1][65].

3. Fichas técnicas de patrones arquitectónicos

En esta sección se presentan las fichas técnicas de los patrones arquitectónicos analizados, describiendo su estructura, características y aplicación en distintos entornos de desarrollo.

3.1. Arquitectura en Capas

Definición: La Arquitectura por Capas es un patrón de diseño de software fundamental que estructura una aplicación descomponiéndola en grupos de sub-tareas, donde cada grupo representa una “capa” con responsabilidades específicas. Este patrón se basa estrictamente en el principio de Separación de Intereses, donde cada capa debe mantener una alta cohesión interna y un bajo acoplamiento con las demás. El flujo de control es generalmente descendente: la capa superior consume los servicios de la capa inmediatamente inferior, garantizando que los cambios en la lógica de negocio o en la base de datos no impacten directamente a la interfaz de usuario. [61], [84], [85]

Descripción de las capas: Aunque el número de capas puede variar según la complejidad del sistema, el modelo estándar para aplicaciones empresariales modernas, especialmente en entornos .NET y Java, define las siguientes capas: [55]

- **Capa de presentación:** Es la interfaz con la que interactúa el usuario final. Su única responsabilidad es mostrar información e interpretar los comandos del usuario (clics, entrada de datos), sin contener lógica de negocio compleja. En aplicaciones de escritorio, esta capa se materializa mediante formularios de Windows (WinForms) o interfaces WPF, mientras que en aplicaciones web corresponde a las vistas HTML/JS que se renderizan en el navegador del cliente [32], [84].
- **Capa de negocio o servicio:** Es el núcleo del sistema donde residen las reglas que gobiernan el proceso (cálculos, validaciones lógicas, flujos de aprobación). Esta capa orquesta la información recibida de la presentación y decide qué hacer con ella. En arquitecturas modernas, se implementa mediante clases de servicio que encapsulan la lógica empresarial y gestionan las transacciones del sistema [13], [61].
- **Capa de acceso a datos:** Su función es abstraer el mecanismo de persistencia. La capa de negocio solicita una operación y la DAL se encarga de traducir esa solicitud al lenguaje de la base de datos, utilizando patrones como Repositorio o DAO (Data Access Object). Esta abstracción permite cambiar el motor de base de datos sin afectar las capas superiores [55], [85].
- **Capa de infraestructura/Base de datos:** Representa el sistema gestor de base de datos (SQL Server, MySQL, PostgreSQL, etc.) donde reside físicamente la información persistente del sistema.

Diagrama conceptual:

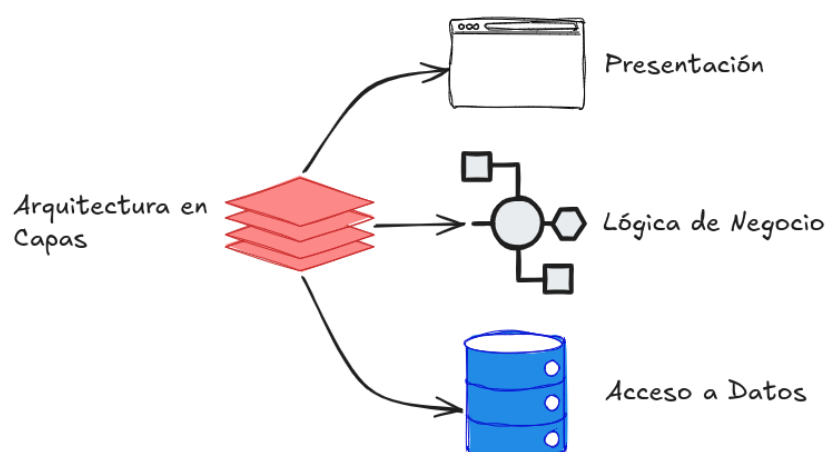


Figura 1: Diagrama de Arquitectura en Pizarra

Aplicación en C#: En el ecosistema de Microsoft, es común separar estas capas mediante bibliotecas de clases (.dll) o carpetas dentro de la solución para mantener el código organizado. La capa de entidad se implementa mediante clases POCO (Plain Old CLR Objects) que representan los objetos del dominio con propiedades públicas y métodos de acceso. La capa de acceso a datos típicamente utiliza Entity Framework Core como ORM, permitiendo trabajar con objetos de C# en lugar de escribir consultas SQL manuales. Esta capa se conecta a SQL Server y ejecuta las operaciones de persistencia de manera transparente [11], [55].

La capa de negocio se implementa mediante clases de servicio que contienen la lógica de validación y las reglas empresariales. Por ejemplo, antes de persistir un registro, el servicio verifica que los datos cumplan con las restricciones del negocio (como validar que un precio no sea negativo en un sistema de inventario). Finalmente, la capa de presentación en WinForms captura los eventos del usuario mediante manejadores de eventos, delega el procesamiento a la capa de negocio y actualiza la interfaz con los resultados obtenidos [13], [20].

Aplicación en Spring Boot: En el desarrollo web con Java y Spring Boot, la inyección de dependencias facilita enormemente la conexión entre estas capas sin generar un acoplamiento fuerte, siguiendo las mejores prácticas de ingeniería de software. La capa de acceso a datos se implementa mediante interfaces Repository que extienden JpaRepository, donde Spring Data JPA genera automáticamente la implementación de los métodos CRUD sin necesidad de escribir código SQL manualmente [42], [84].

La capa de negocio se identifica con la anotación `@Service` y contiene la lógica empresarial del sistema. Aquí se encapsulan las validaciones, cálculos y reglas de negocio, gestionando las transacciones mediante la anotación `@Transactional`. Esta capa actúa como intermediario: el controlador le pasa los datos recibidos desde la web, el servicio aplica las reglas correspondientes y luego invoca al repositorio para persistir la información validada [8], [14].

La capa de presentación en arquitecturas web está representada por los controladores (`@RestController` o `@Controller`), que interceptan las peticiones HTTP (GET, POST, PUT, DELETE) provenientes del navegador. Estos controladores interpretan qué recurso se solicita, extraen los parámetros de la petición y delegan el procesamiento a la capa de servicio, devolviendo finalmente la respuesta adecuada (JSON o una vista renderizada) al

cliente. El navegador actúa como la interfaz gráfica del usuario, renderizando el HTML, CSS y JavaScript enviados desde el servidor [15], [23].

Ventajas:

- **Mantenibilidad:** Dado que las responsabilidades están aisladas, los cambios en una capa (como cambiar el motor de base de datos de SQL Server a PostgreSQL) no afectan a las otras, lo que reduce el esfuerzo de mantenimiento a largo plazo y facilita la evolución del sistema [61], [85].
- **Testabilidad:** Permite realizar pruebas unitarias de forma aislada. Es posible probar la lógica de negocio (BLL) simulando (mocking) la base de datos, lo cual es esencial para asegurar la calidad del software y reducir la propagación de errores entre capas [55], [85].
- **Seguridad:** La capa de presentación no tiene acceso directo a los datos crudos de la base de datos, actuando la capa de negocio como un filtro de seguridad natural que valida y sanitiza la información antes de su persistencia, previniendo ataques como inyección SQL [19], [55].
- **Separación de responsabilidades:** Cada capa tiene un propósito único y bien definido, lo que facilita el trabajo en equipo permitiendo que diferentes desarrolladores trabajen simultáneamente en distintas capas sin generar conflictos de código [47], [84].

Desventajas:

- **Sobrecarga de Rendimiento:** La información debe atravesar múltiples capas antes de llegar a su destino, lo que puede añadir latencia en comparación con una llamada directa en sistemas monolíticos simples. Cada capa introduce un overhead adicional de procesamiento y transferencia de datos [85], [86].
- **Antipatrón “Sinkhole” (Sumidero):** En ocasiones, las capas actúan simplemente como pasamanos de datos sin añadir lógica real, lo que genera código repetitivo (boilerplate) y complejidad innecesaria sin valor agregado. Esto ocurre cuando las capas intermedias no realizan transformaciones significativas sobre los datos [55], [85].
- **Complejidad en el despliegue:** Un cambio menor en la estructura de la base de datos puede requerir actualizaciones en cascada a través de todas las capas (Entidad, DTO, Repositorio, Servicio), lo que aumenta el tiempo de desarrollo y la probabilidad de introducir errores durante el mantenimiento [26], [86].

Escenarios de uso: La arquitectura por capas es especialmente adecuada para sistemas empresariales (ERP/CRM) donde las reglas de negocio son complejas y requieren una clara separación para ser auditables y mantenibles a largo plazo. También es ideal para aplicaciones web modernas donde se separa el frontend (HTML/JS/React) del backend (API REST en Spring Boot o .NET), permitiendo que equipos distintos trabajen en cada capa simultáneamente y facilitando la escalabilidad horizontal del sistema [48], [55], [84].

3.2. Arquitectura de Pizarra (Blackboard)

Definición: La arquitectura en Pizarra es un patrón de diseño arquitectónico utilizado para resolver problemas complejos y no deterministas, en los que la solución se construye de manera incremental mediante la colaboración de diversos módulos especializados (fuentes de conocimiento) que comparten una estructura de datos global centralizada (la pizarra), todo ello orquestado por un componente de control que decide el curso de la ejecución [69], [87].

Descripción estructural: El sistema se organiza en tres componentes fundamentales:

La Pizarra (Blackboard): Es el repositorio central de datos que mantiene el estado actual de la solución, los datos de entrada y la información de control.

Fuentes de Conocimiento (Knowledge Sources - KS): Son módulos independientes especialistas en una parte del problema. No se comunican directamente entre sí; leen la pizarra y, si encuentran datos que pueden procesar, escriben sus resultados parciales en ella.

Controlador: Monitorea los cambios en la pizarra y determina dinámicamente qué Fuente de Conocimiento debe activarse a continuación, basándose en el estado actual de la solución [69], [88].

Diagrama conceptual:

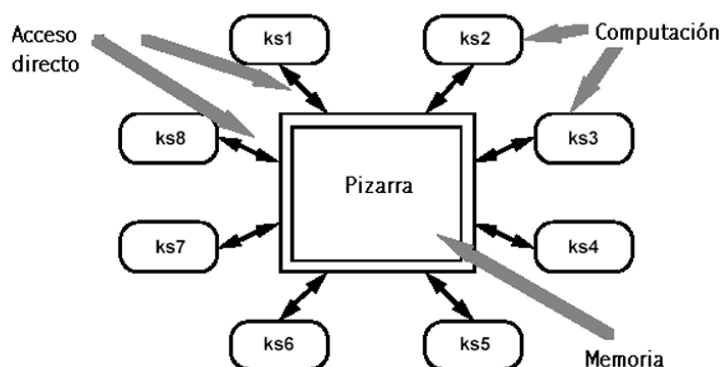


Figura 2: Diagrama de Arquitectura en Pizarra

Aplicación en C#: En el entorno .NET, este patrón es frecuente en motores de reglas o inteligencia artificial para videojuegos. Se implementa habitualmente creando una clase compartida (Singleton) que actúa como la Pizarra, utilizando colecciones seguras para hilos (como *ConcurrentDictionary*) para almacenar los datos. Las Fuentes de Conocimiento se diseñan como clases desacopladas que implementan una interfaz común; el Controlador utiliza reflexión o inyección de dependencias para iterar sobre estas fuentes, evaluando sus precondiciones y ejecutándolas secuencialmente hasta que se alcanza un estado de solución definido [69], [89].

Aplicación en Spring Boot: En el ecosistema de Spring Boot, la Pizarra se modela típicamente como un *Service* (bean con alcance Singleton) que mantiene el modelo de dominio en memoria o caché (ej. Redis). Las Fuentes de Conocimiento son componentes (*@Component*) individuales inyectados en el contexto. El mecanismo de control a menudo se implementa aprovechando el sistema de eventos de Spring (*ApplicationEventPublisher*): cuando un servicio actualiza la pizarra, publica un evento que es capturado asincrónicamente por los componentes interesados, permitiendo una colaboración reactiva y desacoplada sin un flujo de control rígido [88], [90].

Ventajas:

- **Resolución de problemas complejos:** Permite abordar problemas para los que no existe un algoritmo secuencial conocido, facilitando que la solución emerja de la combinación de heurísticas parciales. [69], [87]
- **Flexibilidad y mantenibilidad:** Es sencillo agregar nuevas Fuentes de Conocimiento o mejorar las existentes sin afectar a los demás módulos ni reestructurar el sistema completo [88], [89].

Desventajas:

- **Dificultad de pruebas:** Debido a la naturaleza no determinista y dinámica de la ejecución, reproducir errores y realizar pruebas unitarias aisladas puede ser extremadamente complejo. [69], [87]
- **Cuellos de botella:** La dependencia de un repositorio central de datos puede generar problemas de rendimiento y concurrencia si múltiples fuentes intentan acceder a la pizarra simultáneamente [88], [89].

Escenarios de uso: Se recomienda su aplicación en dominios como el reconocimiento de voz y patrones (donde los datos son ruidosos o incompletos), sistemas de diagnóstico médico, planificación militar, biología molecular y cualquier sistema experto donde se requiera integrar conocimientos diversos para alcanzar una hipótesis de solución [87], [90].

3.3. Arquitectura Dirigida por Eventos

Definición: La Arquitectura Dirigida por Eventos (Event-Driven Architecture, EDA) es un paradigma arquitectónico en el que el flujo del sistema se basa en la producción, detección y consumo de eventos, entendidos como registros inmutables de que ha ocurrido un hecho relevante (por ejemplo, `OrderPlaced` o `PaymentCompleted`) [91], [92], [93]. En este estilo, los productores publican eventos sin conocer a los consumidores, que se suscriben a ellos y reaccionan de manera independiente, favoreciendo un acoplamiento débil y una comunicación asíncrona entre componentes distribuidos [91], [92].

Descripción estructural: Un sistema EDA suele organizarse alrededor de tres roles principales: productores de eventos (servicios o componentes que emiten eventos), un router o broker de eventos (cola, bus o plataforma de streaming) y consumidores de eventos (servicios que procesan los eventos recibidos). [91], [92] La infraestructura de mensajería (por ejemplo, Kafka, RabbitMQ o servicios en la nube) actúa como canal intermedio que desacopla temporal y espacialmente a productores y consumidores, permitiendo escalado independiente, procesamiento paralelo y patrones como pub/sub, colas de trabajo o event streaming [92], [93].

Diagrama conceptual:

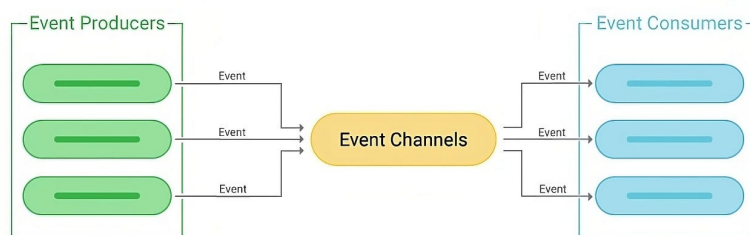


Figura 3: Diagrama de Arquitectura en Pizarra

Aplicación en C#: En el ecosistema .NET, EDA puede implementarse definiendo servicios que publican y consumen mensajes a través de brokers como RabbitMQ o Kafka, usando librerías o frameworks de mensajería (por ejemplo, MassTransit) para modelar eventos como clases C# y suscriptores como handlers que reaccionan a esos eventos [94]. Un escenario típico es un sistema de pedidos donde el servicio de órdenes publica un evento `OrderCreated` en una cola o tópico, y servicios independientes (facturación, inventario, notificaciones) se suscriben y procesan ese evento sin acoplamiento directo al servicio emisor [92], [94].

Aplicación en Spring Boot: En Spring Boot, EDA suele implementarse mediante la publicación y consumo de eventos sobre plataformas como Apache Kafka, utilizando Spring for Apache Kafka o Spring Cloud Stream para definir productores y consumidores basados en anotaciones y bindings hacia tópicos [95]. Por ejemplo, en una arquitectura de microservicios, un servicio de órdenes puede publicar eventos en un tópico de Kafka cuando se crea o actualiza un pedido, mientras otros microservicios (pagos, logística, analítica) consumen esos eventos y actualizan sus propias bases de datos o generan nuevos eventos de manera asíncrona [95].

Ventajas:

- Mejora la escalabilidad y la capacidad de respuesta del sistema al permitir procesamiento asíncrono y paralelo de eventos, adaptándose mejor a picos de carga y

requisitos de tiempo real [91], [92], [93].

- Favorece el desacoplamiento y la flexibilidad, ya que productores y consumidores pueden evolucionar, desplegarse y escalarse de forma independiente, y es posible añadir nuevos consumidores sin modificar los productores existentes [91], [92], [93].

Desventajas:

- Aumenta la complejidad de diseño y operación, especialmente en la gestión de esquemas de eventos, trazabilidad extremo a extremo, manejo de errores y consistencia eventual en sistemas distribuidos [92], [93].
- Dificulta el razonamiento sobre el flujo de negocio y la depuración, ya que las interacciones se encuentran dispersas en múltiples productores, tópicos y consumidores, requiriendo herramientas específicas de observabilidad y monitorización [92], [93].

Escenarios de uso: La EDA es especialmente recomendable en sistemas que requieren procesamiento en tiempo real de grandes volúmenes de eventos, como analítica de flujo, IoT, monitoreo, detección de fraudes o procesamiento de transacciones financieras [92], [93]. También resulta adecuada en arquitecturas de microservicios que necesitan alta resiliencia y desacoplamiento, donde los servicios se comunican principalmente mediante eventos para reducir dependencias sincrónicas y mejorar la tolerancia a fallos [91], [92].

3.4. Arquitectura Peer-to-Peer

Definición: La arquitectura Peer-To-Peer (P2P) es un modelo de sistema distribuido en el que los participantes (nodos o "peers") comparten una parte de sus propios recursos de hardware (capacidad de procesamiento, almacenamiento o ancho de banda) para prestar servicios y contenido directamente a otros participantes, sin la intermediación obligatoria de un servidor centralizado. En este modelo, cada nodo actúa simultáneamente como cliente y servidor (a veces denominado *servent*) [96], [97].

Descripción estructural: El sistema se organiza como una red superpuesta (overlay network) sobre la infraestructura física de red (IP). No existe una jerarquía estricta; todos los nodos son equipotentes en funcionalidad. Existen tres variantes estructurales principales:

- **P2P Puro:** No existe ningún servidor central; el descubrimiento de peers y el enrutamiento se realizan mediante propagación de mensajes (flooding) o tablas hash distribuidas (DHT).
- **P2P Híbrido:** Un servidor central facilita el descubrimiento de nodos (directorio) o la autenticación, pero la transferencia de datos ocurre directamente entre pares.
- **Super-Peers:** Algunos nodos con mayor capacidad actúan como relevos o índices locales para un subconjunto de nodos regulares, equilibrando la carga [97], [98].

Diagrama conceptual:

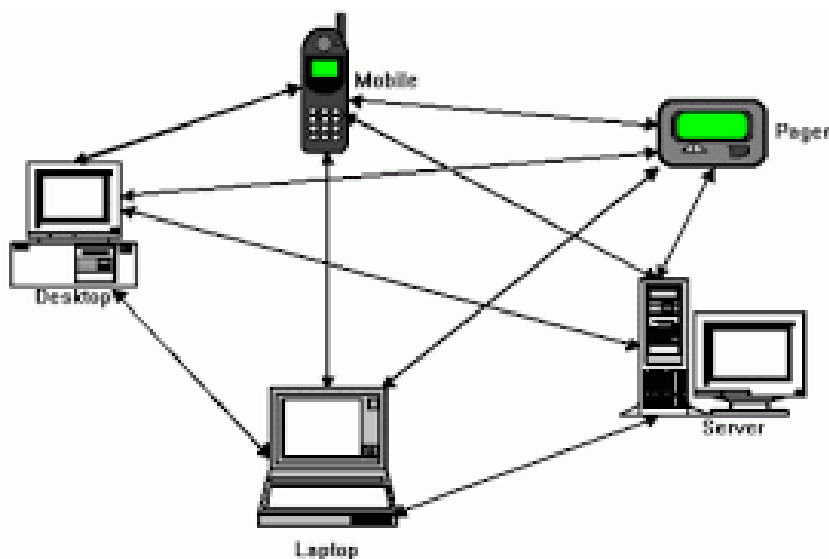


Figura 4: Diagrama de Arquitectura P2P

Aplicación en C#: En el ecosistema .NET, una implementación P2P moderna suele prescindir del obsoleto WCF Peer Channel en favor de SignalR (para el descubrimiento y señalización inicial a través de un hub ligero) combinado con WebRTC o sockets TCP directos para la transferencia de datos real entre pares. Un ejemplo típico es una aplicación de chat o transferencia de archivos donde una aplicación de consola o WPF abre un `TcpListener` en un puerto local para recibir conexiones entrantes de otros pares, mientras usa `TcpClient` para conectarse activamente a direcciones IP conocidas de otros participantes, manteniendo un listado dinámico de vecinos activos [99].

Aplicación en Spring Boot: Aunque Spring Boot es nativo de entornos cliente-servidor, se utiliza en redes P2P (como nodos de blockchain o microservicios descentralizados) exponiendo endpoints REST o gRPC que actúan como interfaces de "servidor" para otros pares. Cada instancia de la aplicación mantiene una lista de direcciones de otros nodos (peers) conocidos. Al iniciar, el nodo consulta a sus vecinos conocidos (gossip protocol) para sincronizar el estado global o descubrir nuevos nodos. Frameworks como Project JXTA (histórico) o integraciones con libp2p sobre Java permiten gestionar la complejidad del descubrimiento y travesía NAT en estas aplicaciones [100].

Ventajas:

- **Escalabilidad orgánica:** La capacidad total del sistema (almacenamiento, ancho de banda, cómputo) aumenta proporcionalmente con cada nuevo nodo que se une, a diferencia del modelo cliente-servidor donde nuevos clientes consumen recursos finitos del servidor [96], [99].
- **Alta tolerancia a fallos:** Al no existir un punto único de falla (en redes puras), la caída de uno o varios nodos no detiene el funcionamiento de la red; el sistema se reorganiza automáticamente para mantener la conectividad [97], [98].

Desventajas:

- **Gestión de seguridad compleja:** Es difícil garantizar la confianza, autenticación y la integridad de los datos cuando no existe una autoridad centralizada que valide a los participantes, facilitando la propagación de contenido malicioso o nodos corruptos (ataques Sybil) [99].
- **Rendimiento impredecible:** La velocidad y disponibilidad dependen de la naturaleza voluntaria de los nodos (churn rate), su ancho de banda doméstico asimétrico y las restricciones de conectividad (firewalls, NAT), lo que no garantiza calidad de servicio (QoS) [97], [98].

Escenarios de uso: Se recomienda en redes de distribución de contenido (CDN) masivo (como actualizaciones de software o streaming de video) para reducir costos de ancho de banda, en redes de criptomonedas y blockchain donde la descentralización es un requisito funcional, y en herramientas de colaboración en tiempo real o comunicación resiliente en entornos donde la infraestructura central no es confiable o existe censura [96], [100].

3.5. Arquitectura Orientada a Servicios (SOA)

Definición: Service-Oriented Architecture (SOA) es un paradigma arquitectónico para desarrollar aplicaciones distribuidas a partir de servicios autocontenidos, interoperables y reutilizables, que se comunican mediante protocolos estandarizados y contratos de servicio bien definidos [72], [101].

Descripción estructural: En una arquitectura SOA, el sistema se organiza alrededor de servicios que encapsulan funcionalidades de negocio y se exponen a través de interfaces descriptivas (por ejemplo, WSDL o descriptores REST), típicamente mediadas por un bus o capa de intermediación que gestiona enrutamiento, seguridad y orquestación [101], [102]. Estos servicios interactúan siguiendo un modelo proveedor-registro-consumidor, donde los consumidores descubren y consumen servicios publicados por distintos proveedores, favoreciendo el bajo acoplamiento y la composición de procesos de negocio más complejos [72], [101].

Diagrama conceptual:

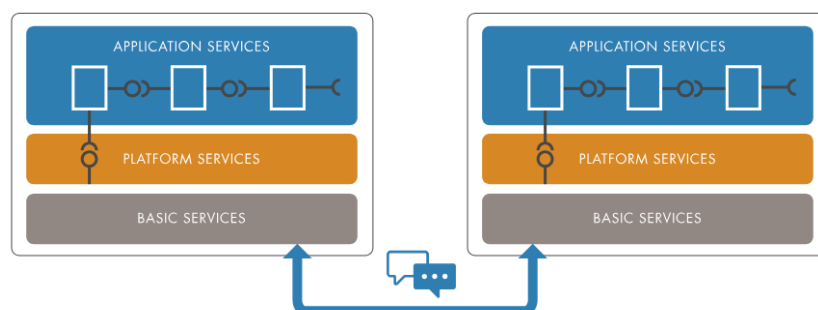


Figura 5: Diagrama de Arquitectura en Pizarra

Aplicación en C#: En el ecosistema .NET, SOA se ha implementado tradicionalmente mediante servicios SOAP con WCF, donde se definen contratos de servicio (interfaces marcadas con `[ServiceContract]` y `[OperationContract]`) que exponen operaciones de negocio a través de endpoints configurados con distintos bindings (HTTP, TCP, etc.). En escenarios más recientes, se pueden construir servicios orientados a recursos con ASP.NET Core Web API o gRPC que siguen principios de SOA (servicios autónomos, contratos explícitos y comunicación estandarizada) integrando sistemas legados o terceros a través de HTTP/JSON o SOAP.

Aplicación en Spring Boot: En el ecosistema Java, Spring Boot permite implementar SOA mediante la construcción de servicios independientes expuestos como RESTful web services, que encapsulan capacidades de negocio y se comunican vía HTTP/JSON, muchas veces complementados con Spring Cloud para descubrimiento, balanceo y configuración centralizada. [103] Estos servicios pueden formar parte de una arquitectura de microservicios, que puede verse como una evolución de SOA a un nivel de granularidad más fino, manteniendo principios de desacoplamiento, reutilización y comunicación estandarizada entre servicios distribuidos [103].

Ventajas:

- Promueve la reutilización de servicios y la interoperabilidad entre plataformas heterogéneas mediante protocolos y estándares abiertos, lo que reduce tiempos de desarrollo e integración en entornos empresariales complejos [101].

- Facilita la escalabilidad y la flexibilidad del sistema al permitir desplegar, versionar y orquestar servicios de forma independiente, adaptando la arquitectura a cambios en los procesos de negocio sin reescribir sistemas monolíticos [72].

Desventajas:

- Introduce complejidad arquitectónica y operativa, debido a la necesidad de gestionar múltiples servicios, contratos, políticas de seguridad y mecanismos de descubrimiento y monitoreo distribuidos [104].
- Puede generar sobrecarga de rendimiento por la comunicación remota entre servicios, especialmente cuando el número de invocaciones o la latencia de red es elevado, lo que requiere optimizaciones adicionales de diseño e infraestructura [104].

Escenarios de uso: SOA se recomienda en organizaciones que integran múltiples sistemas legados y aplicaciones heterogéneas, donde es necesario exponer funcionalidades de negocio como servicios compartidos reutilizables a distintos canales (web, móviles, B2B) [101], [102]. También resulta adecuado en entornos empresariales de gran escala que requieren orquestación de procesos de negocio complejos, alta interoperabilidad y evolución incremental de la arquitectura, sirviendo como base para transicionar hacia arquitecturas de microservicios [72].

3.6. Modelo Vista Controlador (MVC)

Definición: El patrón Modelo-Vista-Controlador (MVC) es un patrón de arquitectura de software ampliamente utilizado en el desarrollo de sistemas informáticos. MVC divide la aplicación en tres partes principales: el modelo, la vista y el controlador, con el objetivo de mejorar aspectos como la seguridad, la interoperabilidad y la usabilidad a lo largo del ciclo de vida del sistema. La amplia adopción de este patrón ha hecho necesaria una evaluación de sus impactos en estos aspectos críticos durante todo el ciclo de desarrollo y mantenimiento de aplicaciones. [53], [58]

Descripción estructural:

- **Modelo (Model):** El Modelo es el componente que contiene y gestiona los datos de la aplicación y su lógica de negocio, siendo responsable de procesar y validar la información, así como de comunicarse con la base de datos para recuperar o guardar datos. En el patrón MVC, el Modelo mantiene la integridad de los datos y abstrae los detalles de almacenamiento para el resto de la aplicación, asegurando que las reglas de negocio se apliquen consistentemente sin importar cómo se presente la información al usuario [53], [54].
- **Controlador (Controller):** El Controlador actúa como intermediario entre el Modelo y la Vista, interpretando las acciones del usuario, procesándolas y determinando qué operaciones del Modelo ejecutar y qué Vista debe mostrarse. El Controlador gestiona la interacción del usuario y coordina la actualización de datos y presentación de los mismos, funcionando como el orquestador principal del flujo de información en la aplicación [53], [54].
- **Vista (View):** La Vista es la parte de la aplicación encargada de presentar la información al usuario. Su función principal es mostrar los datos procesados por el Modelo en una forma comprensible, sin incluir lógica de negocio. La Vista interactúa con el usuario y recibe eventos, pero no modifica los datos por sí misma; simplemente solicita al controlador los cambios necesarios, manteniendo así una clara separación de responsabilidades [53], [54].

Diagrama conceptual:

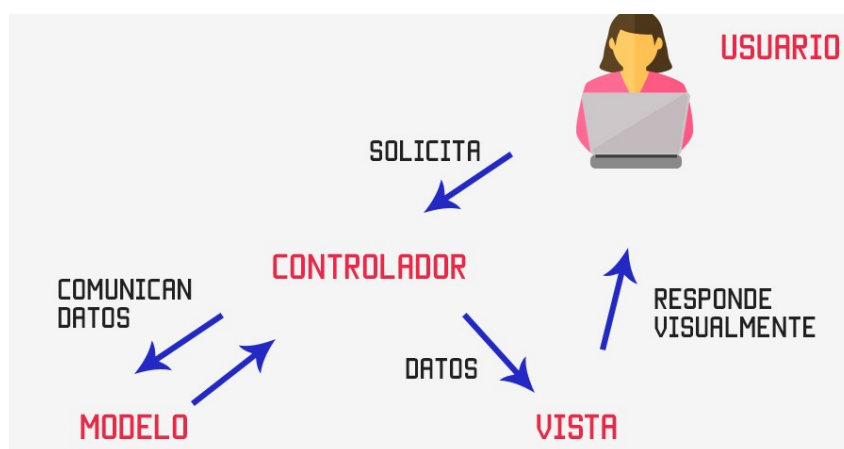


Figura 6: Diagrama de Modelo Vista Controlador

Aplicación en C#: En el ecosistema .NET, el patrón MVC se implementa en aplicaciones de escritorio mediante una clara separación de componentes. El Modelo se construye utilizando clases POCO (Plain Old CLR Objects) que encapsulan la lógica de negocio y las reglas de validación, mientras que el acceso a datos se gestiona típicamente mediante Entity Framework Core o ADO.NET. Estas tecnologías permiten mantener la persistencia de datos completamente abstraída del resto de la aplicación [11], [20], [41].

La Vista en aplicaciones de escritorio se implementa mediante formularios de Windows Forms o ventanas WPF, donde los controles gráficos (botones, cajas de texto, grillas) presentan la información al usuario y capturan sus acciones. El Controlador, aunque no siempre es una clase explícita en aplicaciones de escritorio tradicionales, se materializa en los manejadores de eventos del formulario (event handlers) que responden a las acciones del usuario, invocan los métodos apropiados del Modelo y actualizan la Vista con los resultados obtenidos [20], [30], [32].

En arquitecturas más modernas como .NET MAUI o WPF, el patrón evoluciona hacia MVVM (Model-View-ViewModel), donde se introduce un intermediario adicional que desacopla completamente la Vista del Modelo mediante enlaces de datos bidireccionales (data binding), permitiendo una mayor testabilidad y mantenibilidad del código [30].

Aplicación en Spring Boot: En el desarrollo web con Spring Boot, el patrón MVC se implementa de forma nativa y explícita gracias a la arquitectura del framework. El Modelo se representa mediante clases de entidad anotadas con `@Entity` que mapean directamente a tablas de la base de datos usando JPA (Java Persistence API). El acceso a estos datos se realiza a través de interfaces Repository que extienden `JpaRepository`, donde Spring Data genera automáticamente la implementación de operaciones CRUD sin necesidad de escribir código SQL manualmente [40], [42].

La capa de lógica de negocio se implementa en clases de servicio anotadas con `@Service`, donde se encapsulan las validaciones, cálculos y reglas empresariales. Estos servicios actúan como intermediarios entre los controladores y los repositorios, gestionando las transacciones mediante la anotación `@Transactional` y asegurando la integridad de las operaciones sobre los datos [8], [14].

El Controlador web se define mediante clases anotadas con `@Controller` o `@RestController`, dependiendo de si la aplicación devuelve vistas HTML renderizadas o datos en formato JSON para APIs REST. Estos controladores mapean las URLs a métodos específicos mediante anotaciones como `@GetMapping`, `@PostMapping`, `@PutMapping` y `@DeleteMapping`, interpretando los parámetros de las peticiones HTTP, delegando el procesamiento a los servicios apropiados y determinando la respuesta que debe enviarse al cliente [15], [57], [59].

La Vista en aplicaciones Spring Boot puede implementarse de diversas formas: mediante plantillas del lado del servidor usando motores como Thymeleaf o JSP que generan HTML dinámico, o como aplicaciones de página única (SPA) desarrolladas con frameworks JavaScript como React, Angular o Vue.js que consumen las APIs REST expuestas por los controladores. El navegador web del cliente actúa como el entorno de renderización final de la interfaz de usuario [56], [57].

Ventajas:

- **Separación de responsabilidades:** Permite una separación clara de responsabilidades, ya que el Modelo gestiona los datos, la Vista se encarga de la presentación y el Controlador maneja la lógica de interacción. Esta división facilita la comprensión del código y permite que diferentes aspectos del sistema evolucionen independientemente [54], [58].

- **Mantenibilidad mejorada:** Facilita el mantenimiento del software, debido a que los cambios en la interfaz no afectan directamente la lógica del negocio. Las modificaciones visuales pueden realizarse sin tocar el código del Modelo, y viceversa, reduciendo el riesgo de introducir errores durante el mantenimiento [58], [59].
- **Organización y legibilidad del código:** Mejora la organización del código, haciéndolo más legible y comprensible tanto para el desarrollador original como para nuevos miembros del equipo que deban trabajar en el proyecto [53], [56].
- **Escalabilidad:** Favorece la escalabilidad, permitiendo añadir nuevas funcionalidades sin modificar toda la aplicación. Nuevas vistas pueden agregarse sin alterar el Modelo, y nuevas operaciones de negocio pueden implementarse sin cambiar las vistas existentes [56], [60].
- **Trabajo colaborativo:** Permite el trabajo en equipo, ya que diferentes desarrolladores pueden trabajar simultáneamente en Modelo, Vista y Controlador sin generar conflictos significativos en el código fuente, mejorando la productividad del equipo de desarrollo [57], [58].
- **Testabilidad:** Esta separación de responsabilidades facilita las pruebas, ya que cada componente puede evaluarse por separado mediante pruebas unitarias, sin necesidad de inicializar toda la aplicación o simular la interfaz de usuario [59], [105].

Desventajas:

- **Complejidad para aplicaciones pequeñas:** Puede resultar complejo para aplicaciones pequeñas, ya que requiere mayor cantidad de clases y estructura. El overhead de mantener tres componentes separados puede no justificarse en sistemas simples con pocas funcionalidades [56], [106].
- **Curva de aprendizaje:** Aumenta la curva de aprendizaje para desarrolladores principiantes que deben comprender no solo el lenguaje de programación, sino también cómo interactúan estos tres componentes y cuándo usar cada uno [53], [58].
- **Mayor cantidad de archivos:** Implica una mayor cantidad de archivos, lo que puede dificultar la comprensión inicial del proyecto hasta que el desarrollador se familiarice con la organización y convenciones del patrón [106].
- **Riesgo de mala implementación:** Si no se aplica correctamente, puede generar dependencias innecesarias entre componentes, anulando los beneficios del patrón. Por ejemplo, si la Vista accede directamente al Modelo sin pasar por el Controlador, se rompe la separación de responsabilidades [59], [107].
- **Disciplina de diseño requerida:** Requiere una buena disciplina de diseño para mantener la correcta separación de responsabilidades a lo largo del tiempo, especialmente cuando múltiples desarrolladores trabajan en el proyecto [58], [105].
- **Complejidad en el despliegue:** Comparado con arquitecturas más simples, el despliegue de aplicaciones MVC puede requerir más configuración. Mantener las preocupaciones separadas y asegurarse de que cada parte funcione correctamente en un entorno de producción puede generar complejidad adicional [56].

Escenarios de uso: El patrón MVC surge de la necesidad de crear software robusto que facilita el mantenimiento y reutilización de código a lo largo de todo el ciclo de vida del sistema. La arquitectura Modelo-Vista-Controlador garantiza que cada parte realice una tarea específica, lo cual resulta particularmente valioso en los siguientes contextos:

- **Aplicaciones web empresariales:** Sistemas de ventas, inventarios y gestión de usuarios donde la complejidad de la lógica de negocio justifica la separación en capas y la necesidad de múltiples interfaces para diferentes roles de usuario [56], [57].
- **Aplicaciones de escritorio:** Donde se requiere separar la lógica del negocio de la interfaz gráfica para facilitar el mantenimiento y permitir cambios en la presentación sin afectar las reglas empresariales subyacentes [20], [53].
- **Sistemas académicos:** Plataformas de matrícula, calificaciones y gestión estudiantil que requieren interfaces complejas y lógica de negocio bien estructurada para manejar las reglas académicas institucionales [54].
- **Aplicaciones con interfaces dinámicas:** Donde la presentación cambia frecuentemente según el contexto del usuario, el dispositivo o las preferencias configuradas, beneficiándose de la separación clara entre datos y presentación [56], [60].
- **Proyectos de mediana y gran escala:** Que requieren mantenimiento y evolución continua a largo plazo, donde la inversión inicial en una arquitectura MVC bien estructurada se amortiza con el tiempo mediante menores costos de mantenimiento [58], [105].
- **Aplicaciones desarrolladas con frameworks modernos:** Como Spring Boot, ASP.NET MVC, Ruby on Rails o Django, que implementan MVC de forma nativa y proporcionan herramientas que facilitan seguir este patrón correctamente desde el inicio del proyecto [8], [57].

4. Mapa conceptual comparativo

El presente mapa conceptual relaciona el desarrollo de aplicaciones de escritorio y aplicaciones web, destacando la aplicación de los patrones arquitectónicos estudiados.

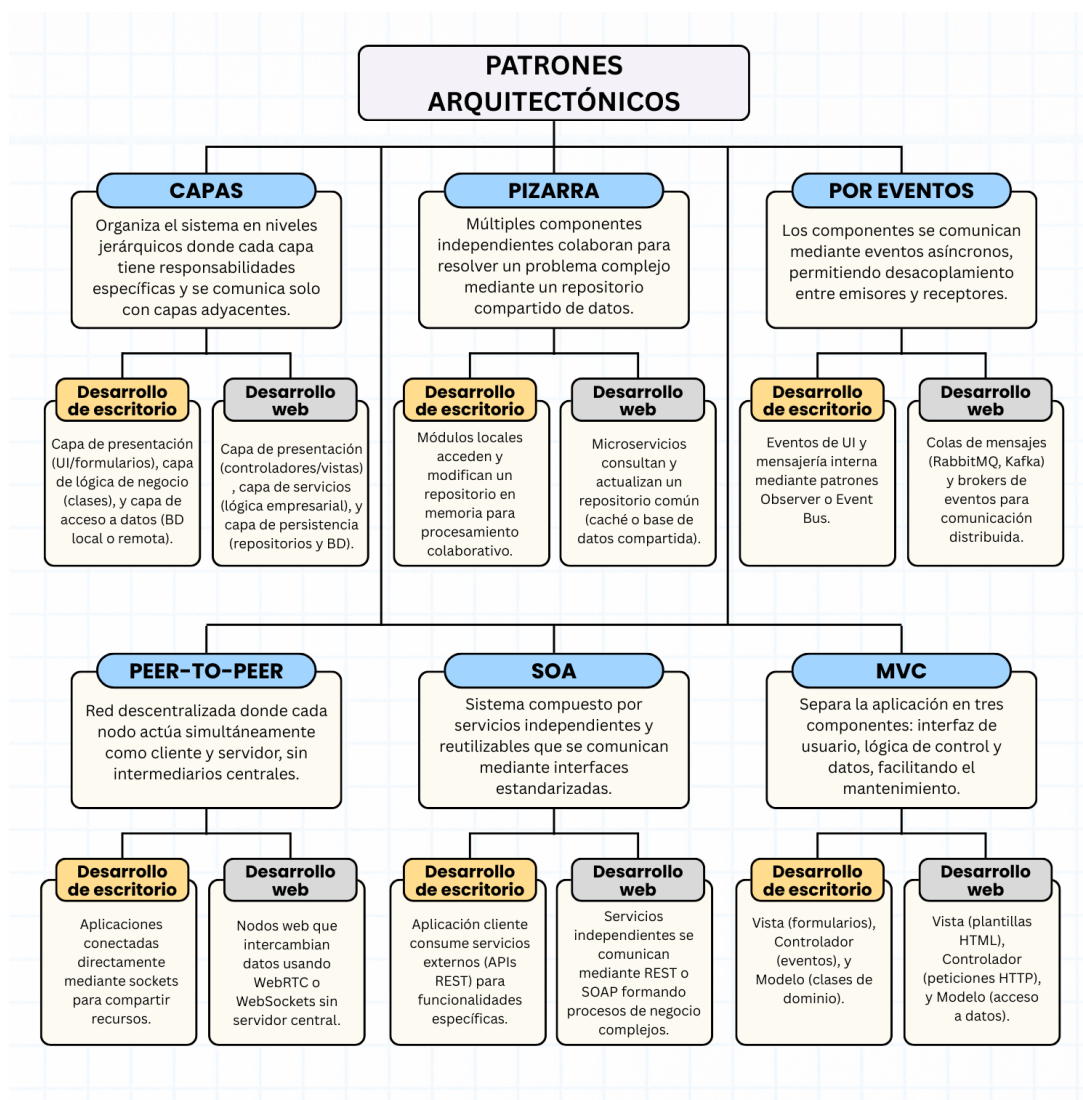


Figura 7: Mapa conceptual comparativo de arquitecturas de software. Realizado en Canva

5. Conclusiones

El desarrollo de esta práctica permitió consolidar la comprensión de los patrones arquitectónicos como elementos fundamentales en el diseño de software, tanto en aplicaciones de escritorio como en aplicaciones web. A través del análisis y la caracterización de arquitecturas como por capas, peer-to-peer, orientada a servicios, dirigida por eventos, pizarra y el modelo MVC, se evidenció que la arquitectura no solo define la estructura técnica de un sistema, sino que también condiciona su mantenibilidad, escalabilidad y capacidad de evolución a lo largo del tiempo.

Asimismo, la elaboración del glosario técnico y el análisis comparativo entre los ámbitos de escritorio y web favorecieron el uso adecuado de la terminología propia de la ingeniería

de software, permitiendo distinguir cómo un mismo concepto puede variar en su aplicación según el contexto tecnológico. Este ejercicio fortaleció la capacidad de análisis crítico y promovió una visión más integral del desarrollo de software moderno, donde la modularidad, el desacoplamiento y la correcta comunicación entre componentes son aspectos clave.

Finalmente, la práctica reforzó la importancia de seleccionar patrones arquitectónicos acordes a los requisitos y características de cada sistema, evitando soluciones genéricas que puedan comprometer la calidad del software. Comprender estos patrones y su aplicación práctica constituye una base sólida para el desarrollo de sistemas robustos, eficientes y alineados con las buenas prácticas de la ingeniería de software, lo cual resulta esencial para la formación profesional en el área.

Referencias

- [1] L. Bass, P. Clements y R. Kazman, *Software Architecture in Practice*, 3.^a ed. Upper Saddle River, NJ: Addison-Wesley, 2013. dirección: <https://ptgmedia.pearsoncmg.com/images/9780321815736/samplepages/0321815734.pdf>.
- [2] I. Sommerville, *Ingeniería de Software*, 9.^a ed. Madrid, España: Pearson, 2011. dirección: https://gc.scalahed.com/recursos/files/r161r/w25469w/ingdelsoftwarelibro9_compressed.pdf.
- [3] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA: Addison-Wesley, 2003.
- [4] R. C. Martin, *About This E-Book*. 2014.
- [5] P. C. Clements, *Software Architecture in Practice*. Software Engineering Institute, 2014.
- [6] K. E. Kendall, *Análisis y Diseño de Sistemas*. 2011. dirección: www.xlibros.com.
- [7] F. de Asís López Fuentes, *Sistemas Distribuidos*. 2015. dirección: http://dccd.cua.uam.mx/libros/archivos/03IXStream_sistemas_distribuidos.pdf.
- [8] L. Spilca, *Spring Start Here*. 2021. dirección: <https://abitap.com/wp-content/uploads/2024/02/spring-start-here-learn-what-you-need-and-learn-it-well-2021-laurentiu-spilca.pdf>.
- [9] *PE Tema12 2023*, 2023. dirección: <https://es.scribd.com/document/730128798/pe-tema12-2023>.
- [10] G. E. Ingeniería Del Software, “Proyecto de Fin de Grado,” Tesis de mtría., Universidad Politécnica de Madrid, Escuela Técnica Superior de Ingeniería de Sistemas Informáticos, 2025. dirección: https://oa.upm.es/38731/1/TFG_Federico_Gutierrez_Faraoni.pdf.
- [11] Microsoft, *Información general de Entity Framework Core - EF Core*, 2025. dirección: <https://learn.microsoft.com/es-es/ef/core/>.
- [12] C. Software, “Proyecto de Titulación,” Tesis de mtría., Escuela Superior Politécnica de Chimborazo, Facultad de Informática y Electrónica, 2025. dirección: <https://dspace.esPOCH.edu.ec:8080/server/api/core/bitstreams/5d774c18-be2f-4c5d-978f-c7060377bdf0/content>.
- [13] B. W. M. R. Cesar de la Torre, *.NET Microservices Architecture for Containerized .NET Applications*, 2025. dirección: <https://learn.microsoft.com/es-es/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/net-core-microservice-domain-model>.
- [14] C. E. Vera Barzola, “Proyecto de Titulación Previo a la Obtención del Título,” Tesis de mtría., Universidad Estatal del Sur de Manabí, Facultad de Ciencias Técnicas, Carrera de Tecnologías de la Información, 2025. dirección: <https://repositorio.unesum.edu.ec/bitstream/53000/4808/1/Vera%20Barzola%20Cynthia%20Elizabeth.pdf>.
- [15] Spring Framework, *Spring Web MVC*, 2025. dirección: <https://docs.spring.io/spring-framework/reference/web/webmvc.html>.

-
- [16] W. Cunningham y R. Johnson, *UML y Patrones*, 2.^a ed. 2006. dirección: https://upload.wikimedia.org/wikipedia/commons/a/a8/UML_y_Patrones_-_Larman_2_Edicion.pdf.
 - [17] T. Real, *Análisis y Diseño Orientado a Objetos*, 2025. dirección: https://www.academia.edu/35650562/An%C3%A1lisis_y_dise%C3%B1o_orientado_a_objetos.
 - [18] M. J. Price, *C# 12 and .NET 8: Modern Cross-Platform Development Fundamentals*. 2023, pág. 826.
 - [19] *Spring Security in Action*, 2025. dirección: <https://studylib.net/doc/27897532/spring-security-in-action>.
 - [20] A. Troelsen y P. Japikse, *Pro C# 10 with .NET 6*. Apress, 2022. dirección: <https://dl.ebooksworld.ir/books/Pro.CSharp.10.with.NET.6.Andrew.Troelsen.Phil.Japikse.Apress.9781484278680.EBooksWorld.ir.pdf>.
 - [21] *Spring Boot: Up and Running*. O'Reilly Media, Inc, 2021.
 - [22] Microsoft, *Control y Generación de Eventos - .NET*, 2025. dirección: <https://learn.microsoft.com/es-es/dotnet/standard/events/>.
 - [23] *Arquitectura de Microservicios con Spring Boot*, 2025. dirección: <https://es.scribd.com/document/423812839/Arquitectura-Basados-en-Microservicios-Spring-Cloud-y-Netflix-Oss-Para-A-0>.
 - [24] Microsoft, *Propiedades - C#*, 2025. dirección: <https://learn.microsoft.com/es-es/dotnet/csharp/programming-guide/classes-and-structs/properties>.
 - [25] H. Schildt, *Java: The Complete Reference*. 2025. dirección: <https://www.sietk.org/downloads/javabook.pdf>.
 - [26] Microsoft, *Microservicios de .NET. Arquitectura para Aplicaciones .NET en Contenedor*, 2025. dirección: <https://learn.microsoft.com/es-es/dotnet/architecture/microservices/>.
 - [27] M. Richards y N. Ford, *Fundamentals of Software Architecture: A Modern Engineering Approach*. 2025. dirección: <https://books.google.es/books?id=yC10EQAAQBAJ>.
 - [28] J. Albahari, *C# 10 in a Nutshell: The Definitive Reference*. O'Reilly, 2025. dirección: <https://dl.ebooksworld.ir/books/CSharp.10.in.a.Nutshell.Joseph.Albahari.OReilly.9781098121952.EBooksWorld.ir.pdf>.
 - [29] Spring Framework, *DispatcherServlet*, 2025. dirección: <https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-servlet.html>.
 - [30] Microsoft, *Modelo-Vista-Modelo de Vista - .NET*, 2025. dirección: <https://learn.microsoft.com/es-es/dotnet/architecture/maui/mvvm>.
 - [31] “MVC Design Pattern for the Multi Framework Distributed Applications Using XML, Spring and Struts Framework,” 2025. dirección: https://www.researchgate.net/publication/49619227_MVC_Design_Pattern_for_the_multi_framework_distributed_applications_using_XML_spring_and_struts_framework.
 - [32] Microsoft, *Qué es Windows Forms - Windows Forms*, 2025. dirección: <https://learn.microsoft.com/es-es/dotnet/desktop/winforms/overview/>.

-
- [33] Microsoft, *Implementación y Seguridad de ClickOnce - Visual Studio*, 2025. dirección: <https://learn.microsoft.com/es-es/visualstudio/deployment/clickonce-security-and-deployment>.
 - [34] A. S. Mohammed, V. R. Saddi, S. K. Gopal, S. Dhanasekaran y M. S. Naruka, “AI-Driven Continuous Integration and Continuous Deployment in Software Engineering,” en *2024 2nd International Conference on Disruptive Technologies, ICDT 2024*, 2024, págs. 531-536. DOI: 10.1109/ICDT61202.2024.10489475.
 - [35] Microsoft, *Creación y Configuración de un Modelo: EF Core*, 2025. dirección: <https://learn.microsoft.com/es-es/ef/core/modeling/>.
 - [36] F. Delporte, *Getting Started with Java on the Raspberry Pi*. 2020. dirección: <https://es.scribd.com/document/642429155/Delporte-F-Getting-started-with-Java-on-the-Raspberry-Pi-2020>.
 - [37] Microsoft, *¿Qué son las Soluciones y Proyectos de Visual Studio?* 2025. dirección: <https://learn.microsoft.com/es-es/visualstudio/ide/solutions-and-projects-in-visual-studio>.
 - [38] F. Gutierrez, “Spring Boot Internals,” en *Pro Spring Boot 3*, Berkeley, CA: Apress, 2024, págs. 39-83. DOI: 10.1007/978-1-4842-9294-5_2.
 - [39] D. Nesteruk, *Design Patterns in .NET 6: Reusable Approaches in C# and F# for Object-Oriented Software Design*, Third. 2022. DOI: 10.1007/978-1-4842-8245-8.
 - [40] R. Harrop, C. Schaefer y C. Ho, *Pro Spring 5: An In-Depth Guide to the Spring Framework and Its Tools*, Fifth. 2017. DOI: 10.1007/978-1-4842-2808-1.
 - [41] J. P. Smith, *Entity Framework Core in Action*, Second. 2021. dirección: <https://books.google.com.ec/books?id=sJsvEAAAQBAJ>.
 - [42] Spring Data, *Spring Data JPA*, 2025. dirección: <https://docs.spring.io/spring-data/jpa/reference/index.html>.
 - [43] Microsoft, *Diseño de Validaciones en la Capa de Modelo de Dominio - .NET*, 2025. dirección: <https://learn.microsoft.com/es-es/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/domain-model-layer-validations>.
 - [44] Microsoft, *Pintura Personalizada para un Control - Windows Forms*, 2025. dirección: <https://learn.microsoft.com/es-es/dotnet/desktop/winforms/controls/custom-painting-drawing>.
 - [45] Microsoft, *Bibliotecas de Clases de .NET*, 2025. dirección: <https://learn.microsoft.com/es-es/dotnet/standard/class-libraries>.
 - [46] M. Deinum, D. Rubio y J. Long, *Spring 6 Recipes: A Problem-Solution Approach to Spring Framework*, Fifth. 2023, págs. 1-753. DOI: 10.1007/978-1-4842-8649-4.
 - [47] *Clean Architecture .NET*, 2025. dirección: <https://es.scribd.com/document/660549582/CleanArchNET>.
 - [48] Spring Framework, *Spring Framework Overview*, 2025. dirección: <https://docs.enterprise.spring.io/spring-framework/reference/overview.html>.
 - [49] Microsoft, *Validación de Entrada de Usuario - Windows Forms*, 2025. dirección: <https://learn.microsoft.com/es-es/dotnet/desktop/winforms/input-keyboard/validation>.

-
- [50] Jakarta EE, *Especificación de Validación de Jakarta Bean*, 2025. dirección: <https://jakarta.ee/specifications/bean-validation/3.0/jakarta-bean-validation-spec-3.0.html>.
 - [51] Spring Framework, *Descripción General del Contenedor*, 2025. dirección: <https://docs.spring.io/spring-framework/reference/core/beans/basics.html>.
 - [52] Microsoft, *Administrar Referencias en un Proyecto - Visual Studio*, 2025. dirección: <https://learn.microsoft.com/es-es/visualstudio/ide/managing-references-in-a-project>.
 - [53] E. Lumba y A. Waworuntu, "Implementation of Model View Controller Architecture in Object Oriented Programming Learning," *International Journal of New Media Technology*, vol. 8, n.º 2, pág. 102, 2021.
 - [54] R. N. Thakur y U. S. Pandey, "The Role of Model-View Controller in Object Oriented Software Development," *Nepal Journal of Multidisciplinary Research*, 2019.
 - [55] Microsoft Corporation, *Architecting Modern Web Applications with ASP.NET Core and Azure*, 2025. dirección: <https://learn.microsoft.com/es-es/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures>.
 - [56] A. Amit Damle y S. Kumar Goel, "Modern Web Development: The Role and Evolution of MVC Architecture," *An ISO*, vol. 13, n.º 6, pág. 12 044, 2024. DOI: 10.15680/IJIRSET.2024.1306239.
 - [57] D. P. Pop y A. Altar, "Designing an MVC model for rapid web application development," en *Procedia Engineering*, Elsevier Ltd, 2014, págs. 1172-1179. DOI: 10.1016/j.proeng.2014.03.106.
 - [58] F. Enríquez, S. Fierro, B. Flores, D. Imbaquingo Esparza y J. Michelena, "Impacto del patrón modelo vista controlador (MVC) en la seguridad, interoperabilidad y usabilidad de un sistema informático durante su ciclo de vida," *EASI: Ingeniería y Ciencias Aplicadas en la Industria*, vol. 2, n.º 1, págs. 11-16, jul. de 2023. DOI: 10.53591/easi.v2i1.2043.
 - [59] M. Kumar y V. Choppella, "Enhancing MVC architecture pattern description using its System of Systems model," en *ACM International Conference Proceeding Series*, Association for Computing Machinery, feb. de 2024. DOI: 10.1145/3641399.3641410.
 - [60] X. Zhang y Z. Zhang, "Research on Web Development Paradigm based on MVC Pattern and Agile Development Mode," 2016.
 - [61] I. Sommerville, *Ingeniería del Software*, 7.^a ed. 2005, pág. 687. dirección: <http://books.google.com/books?id=gQWd49zSut4C&pgis=1>.
 - [62] R. S. Pressman, *Software Engineering: A Practitioner's Approach*, 8.^a ed. New York, NY: McGraw-Hill Education, 2014.
 - [63] IEEE, "IEEE Standard 1471-2000: Recommended Practice for Architectural Description of Software-Intensive Systems," Institute of Electrical y Electronics Engineers, New York, NY, Standard, 2000.
 - [64] A. S. Tanenbaum y M. van Steen, *Distributed Systems*, 3rd. Amsterdam, Netherlands: Maarten van Steen, 2017, Previously published by Pearson Education, Inc., ISBN: 978-15-430573-8-6. dirección: <https://www.distributed-systems.net>.

-
- [65] G. Coulouris, J. Dollimore, T. Kindberg y G. Blair, *Distributed Systems: Concepts and Design*, 5th. Boston, MA: Pearson Education, 2012, ISBN: 978-0-13-214301-1. dirección: <https://www.pearson.com/en-us/subject-catalog/p/distributed-systems-concepts-and-design/P200000003160>.
 - [66] M. Richards, *Software Architecture Patterns*, 2nd. Sebastopol, CA: O'Reilly Media, 2022, ISBN: 978-1-098-13427-3. dirección: <https://www.oreilly.com/library/view/software-architecture-patterns/9781098134280/>.
 - [67] S. Newman, *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith*. Sebastopol, CA: O'Reilly Media, 2019, ISBN: 978-1-492-04784-8. dirección: <https://www.oreilly.com/library/view/monolith-to-microservices/9781492047834/>.
 - [68] G. Hohpe y B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Boston, MA: Addison-Wesley Professional, 2003, ISBN: 978-0-321-20068-6. dirección: <https://www.enterpriseintegrationpatterns.com>.
 - [69] F. Buschmann, R. Meunier, H. Rohnert, P. Sommerlad y M. Stal, *Pattern-Oriented Software Architecture, Volume 1: A System of Patterns*. Chichester, UK: John Wiley & Sons, 1996, ISBN: 978-0-471-95869-7. dirección: <https://www.wiley.com/en-us/Pattern+Oriented+Software+Architecture>.
 - [70] D. D. Corkill, "Blackboard Systems," *AI Expert*, vol. 6, n.º 9, págs. 40-47, sep. de 1991. dirección: <http://mas.cs.umass.edu/paper/218>.
 - [71] C. Szyperski, *Component Software: Beyond Object-Oriented Programming*, 2nd. Boston, MA: Addison-Wesley Professional, 2002, ISBN: 978-0-201-74572-0. dirección: <https://dl.acm.org/doi/book/10.5555/515228>.
 - [72] T. Erl, *Service-Oriented Architecture: Concepts, Technology, and Design*. Upper Saddle River, NJ: Prentice Hall, 2005, ISBN: 978-0-13-185858-9. dirección: <https://www.informit.com/store/service-oriented-architecture-concepts-technology-and-9780131858589>.
 - [73] B. Meyer, *Object-Oriented Software Construction*, Introduces Design by Contract principles for formal interface specifications, Upper Saddle River, NJ, 1997.
 - [74] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA: Addison-Wesley Professional, 2003, ISBN: 978-0-321-12521-7. dirección: <https://www.domainlanguage.com/ddd/>.
 - [75] R. C. Martin, *Design Principles and Design Patterns*. Objectmentor, 2000, Introduces SOLID principles including Dependency Inversion Principle. dirección: https://www.objectmentor.com/resources/articles/Principles_and_Patterns.pdf.
 - [76] N. Ford, R. Parsons y P. Kua, *Building Evolutionary Architectures: Support Constant Change*, 1st. Sebastopol, CA: O'Reilly Media, 2017, ISBN: 978-1-491-98636-3. dirección: <https://www.oreilly.com/library/view/building-evolutionary-architectures/9781491986356/>.
 - [77] H. P. Breivold, I. Crnkovic y P. Eriksson, "Software Architecture Evolution through Evolvability Analysis," *Journal of Systems and Software*, vol. 85, n.º 11, págs. 2574-2592, 2012. DOI: 10.1016/j.jss.2012.06.003. dirección: <https://www.sciencedirect.com/science/article/abs/pii/S0164121212001637>.

-
- [78] M. Shaw y D. Garlan, “An Introduction to Software Architecture,” *Advances in Software Engineering and Knowledge Engineering*, 1994, Introduces foundational architectural patterns including pipes and filters, data flow architecture. dirección: https://cs.uwaterloo.ca/~a78khan/cs446/lectures/2011_05-may_16_IntroductionToArchitecture-GarlanAndShaw.pdf.
 - [79] G. Young, “Event Sourcing,” *GOTO Conferences*, 2014, Influential presentation on event sourcing and event-driven architecture patterns. dirección: <https://www.youtube.com/watch?v=8JKjvY4etTY>.
 - [80] R. Kazman, D. Garlan y L. Bass, “Toward Deriving Software Architectures from Quality Attributes,” en *Proceedings of the Software Engineering Institute Technical Report CMU/SEI-94-TR-008*, Pittsburgh, PA, 1994. dirección: https://www.sei.cmu.edu/documents/1109/1994_005_001_16301.pdf.
 - [81] C. Richardson, *Microservices Patterns: With Examples in Java*. Shelter Island, NY: Manning Publications, 2018, ISBN: 978-1-617-29409-9. dirección: <https://microservices.io/book>.
 - [82] D. C. Schmidt, M. Stal, H. Rohnert y F. Buschmann, “Patterns, Frameworks, and Middleware: Their Synergistic Relationships,” en *Proceedings of the 25th International Conference on Software Engineering (ICSE 2003)*, Portland, OR, 2003, págs. 694-704. dirección: <https://www.dre.vanderbilt.edu/~schmidt/PDF/ICSE-03.pdf>.
 - [83] S. M. Thampi y R. Jose, “Introduction to Distributed Systems,” *arxiv*, 2009. dirección: <https://arxiv.org/pdf/0911.4395.pdf>.
 - [84] R. S. Pressman, *Ingeniería del Software: Un Enfoque Práctico*. 2021. dirección: <https://profesorezequielruizgarcia.wordpress.com/wp-content/uploads/2015/01/ingenieria-del-software-un-enfoque-practico-roger-s-pressman.pdf>.
 - [85] M. Richards y N. Ford, *Fundamentals of Software Architecture: An Engineering Approach*. 2020, pág. 400. dirección: <https://books.google.com.ec/books?id=xa7MDwAAQBAJ>.
 - [86] V. T. Colombero, J. P. Estelles, L. Gallegos y P. Lopez, “Monolitos vs. Microservicios en Arquitectura de Software: Perspectivas para un Desarrollo Eficiente,” 2024.
 - [87] D. Garlan y M. Shaw, “An Introduction to Software Architecture,” Carnegie Mellon University, inf. téc. CMU-CS-94-166, 1994. dirección: <https://www.cs.cmu.edu/~able/publications/papers/intro-softarch.pdf>.
 - [88] Curate Partners. “Understanding the Blackboard Architectural Pattern.” Accedido: 20-12-2025. dirección: <https://curatepartners.com/tech-skills-tools-platforms/understanding-the-blackboard-architectural-pattern/>.
 - [89] Microsoft Learn, *Cloud Design Patterns: Architectural patterns*, Guía de referencia para arquitectura en .NET, 2024. dirección: <https://learn.microsoft.com/en-us/azure/architecture/patterns/>.
 - [90] A. N. M. B. Rahman. “The Blackboard Pattern for Autonomous Navigation.” Ejemplo aplicado en entorno Java/Spring. dirección: <https://dzone.com/articles/the-blackboard-pattern-for-autonomous-navigation>.

-
- [91] Amazon Web Services. “Event-Driven Architecture.” Accedido: 20-12-2025. dirección: <https://aws.amazon.com/event-driven-architecture/>.
 - [92] Confluent. “Event-Driven Architecture (EDA): A Complete Introduction.” Accedido: 20-12-2025. dirección: <https://www.confluent.io/learn/event-driven-architecture/>.
 - [93] Future Processing. “What is event-driven architecture? A complete guide.” Accedido: 20-12-2025. dirección: <https://www.future-processing.com/blog/event-driven-architecture-guide/>.
 - [94] C. Maze. “Event-Driven Architecture in C#.” Accedido: 20-12-2025. dirección: <https://code-maze.com/csharp-event-driven-architecture/>.
 - [95] A. J. Catambay, “Event-driven Architectures on Apache Kafka with Spring Boot,” *YouTube / GitHub accompanying material*, 2020, Accedido: 20-12-2025. dirección: <https://github.com/ajsaavedra/ProductOrdersAnalytics-kafka-demo>.
 - [96] R. Schollmeier, “A Definition of Peer-to-Peer Networking for the Classification of Peer-to-Peer Architectures and Applications,” en *Proceedings of the First International Conference on Peer-to-Peer Computing*, 2001, págs. 101-102. DOI: 10.1109/P2P.2001.990434.
 - [97] R. Steinmetz y K. Wehrle, *Peer-to-Peer Systems and Applications*. Springer, 2005, ISBN: 978-3540291923. DOI: 10.1007/11530657.
 - [98] GeeksforGeeks. “Peer-to-Peer (P2P) Architecture: Comprehensive Guide.” dirección: <https://www.geeksforgeeks.org/system-design/peer-to-peer-p2p-architecture/>.
 - [99] IDC Technologies, *Peer-to-Peer (P2P): Advantages and Disadvantages*, Análisis técnico de pros y contras, 2024. dirección: https://www.idc-online.com/technical_references/pdfs/data_communications/Peer_to_Peer_Advantages_and_Disadvantages.pdf.
 - [100] A. Verma. “Building a P2P file-sharing network in Java without a central server.” Caso de estudio de implementación práctica. dirección: https://www.linkedin.com/posts/alok-verma-b158b0259_java-p2p-networking-activity-7383903676643364864F8C-.
 - [101] M. P. Papazoglou, “Service-oriented computing: concepts, characteristics and directions,” *Proceedings of the Fourth International Conference on Web Information Systems Engineering*, págs. 3-12, 2003. DOI: 10.1109/WISE.2003.1254461.
 - [102] K. Domdouzis et al., “Application of a new service-oriented architecture (SOA) for RF-based asset location tracking,” *International Journal of Distributed Systems and Technologies*, vol. 7, n.º 2, págs. 1-21, 2016. DOI: 10.4018/IJDST.2016040101. dirección: <https://shura.shu.ac.uk/12183/>.
 - [103] S. Singh et al., “Microservices with Spring Boot: Simplifying Distributed Systems,” *International Journal of Future Generation Communication and Networking*, págs. 1-10, 2024. dirección: <https://www.ijfmr.com/papers/2024/5/28930.pdf>.
 - [104] Z. M. Aljazzaf, “Trust-based Service-Oriented Architecture,” *International Journal of Advanced Computer Science and Applications*, vol. 7, n.º 1, págs. 417-424, 2016. DOI: 10.14569/IJACSA.2016.070156. dirección: <https://www.sciencedirect.com/science/article/pii/S1319157816300064>.

-
- [105] S. Neca, *Exploring The Model-View-Controller (MVC) Architecture: A Broad Analysis of Market and Technological Applications*, abr. de 2024. DOI: 10.20944/preprints202404.1860.v1.
 - [106] M. Babiuch y P. Foltyn, “Implementation of a Universal Framework Using Design Patterns for Application Development on Microcontrollers,” *Sensors*, vol. 24, n.º 10, mayo de 2024. DOI: 10.3390/s24103116.
 - [107] G. Paolone, R. Paesani, M. Marinelli y P. Di Felice, “Empirical Assessment of the Quality of MVC Web Applications Returned by xGenerator,” 2021. DOI: 10.3390/computers.